

4.4 常用组合逻辑电路模块

4.4.1 编码器

4.4.2 译码器/数据分配器

4.4.3 数据选择器

4.4.4 数值比较器

4.4.5 算术运算电路

4.4 常用组合逻辑电路模块

4.4.1 编码

器

1、编码器 (Encoder) 的定义与分类

编码：赋予二进制代码特定含义的过程称为编码。

如：8421BCD 码中，用 1000 表示数字 8

如：ASCII 码中，用 1000001 表示字母 A 等

编码器：具有编码功能的逻辑电路。

1、编码器 (Encoder) 的定义与分类

编码器的逻辑功能:

能将每一个编码输入信号变换为不同的二进制的代码输出。

- ◆ **如 BCD 编码器: 将 10 个编码输入信号分别编成 10 个 4 位码输出。**
- ◆ **如 8 线 -3 线编码器: 将 8 个输入的信号分别编成 8 个 3 位二进制数码输出。**

1、编码器 (Encoder) 的定义与分类

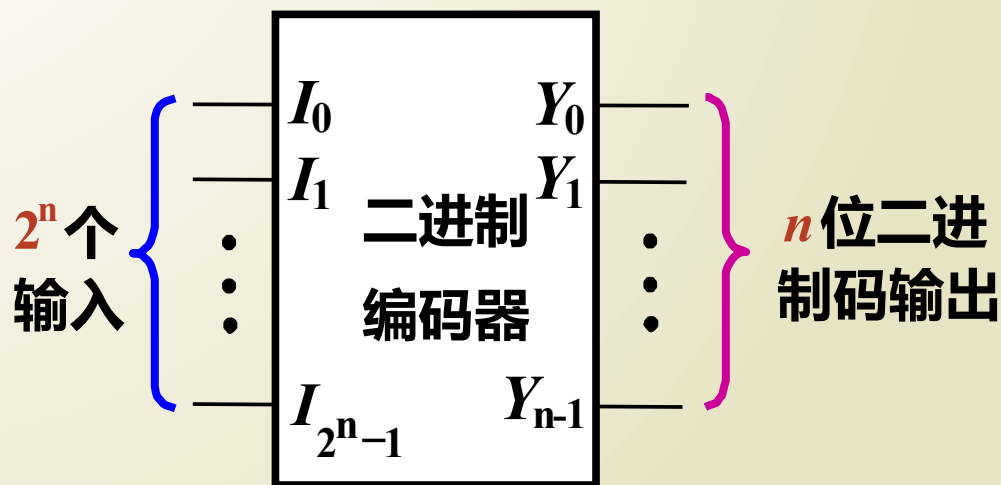
编码器的分类：普通编码器和优先编码器。

- ◆ **普通编码器：任何时候只允许输入一个有效编码信号，否则输出就会发生混乱。**
- ◆ **优先编码器：允许同时输入两个以上的有效编码信号。当同时输入几个有效编码信号时，优先编码器能按预先设定的优先级别，只对其中优先权最高的一个进行编码。**

2、编码器的工作原理

普通二进制编码器

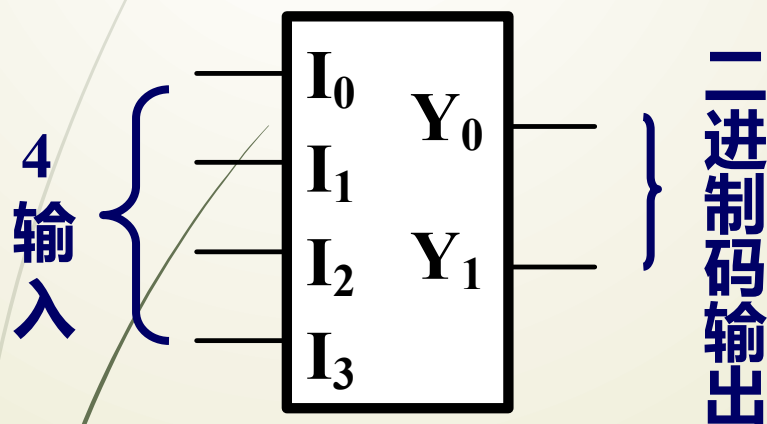
二进制编码器的结构框图



2、普通编码器

(1) 4 线—2 线普通二进制编码器（设计）

(a) 逻辑框图



$$Y_1 = \bar{I}_0 \bar{I}_1 I_2 \bar{I}_3 + \bar{I}_0 \bar{I}_1 \bar{I}_2 I_3$$

$$Y_0 = \bar{I}_0 I_1 \bar{I}_2 \bar{I}_3 + \bar{I}_0 \bar{I}_1 \bar{I}_2 I_3$$

该表达式是否可以再简化？

(2) 逻辑功能表

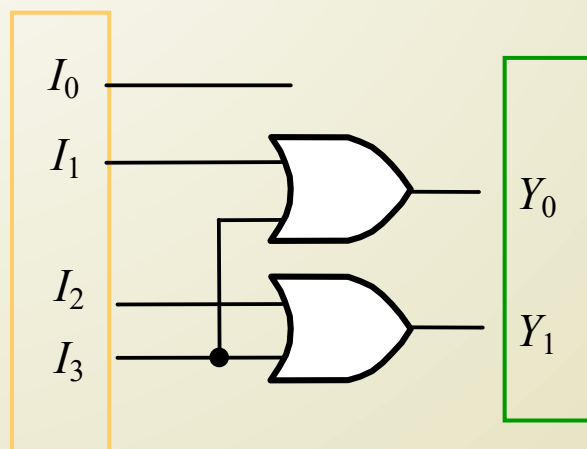
I_0	I_1	I_2	I_3	Y_1	Y_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

编码器的输入为高电平有效。

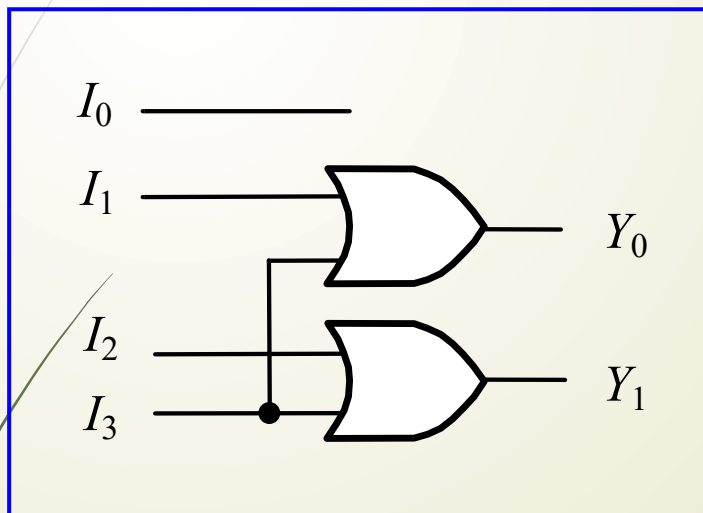
上述是将输入的其它 12 种组合对应的输出看做 0。如果看做无关项，则表达式为

$$Y_1 = I_2 + I_3$$

$$Y_0 = I_1 + I_3$$



若有 2 个以上的输入为有效信号？



当只有 I_3 为 1 时,

$Y_1 Y_0 = ?$ $Y_1 Y_0 = 11$

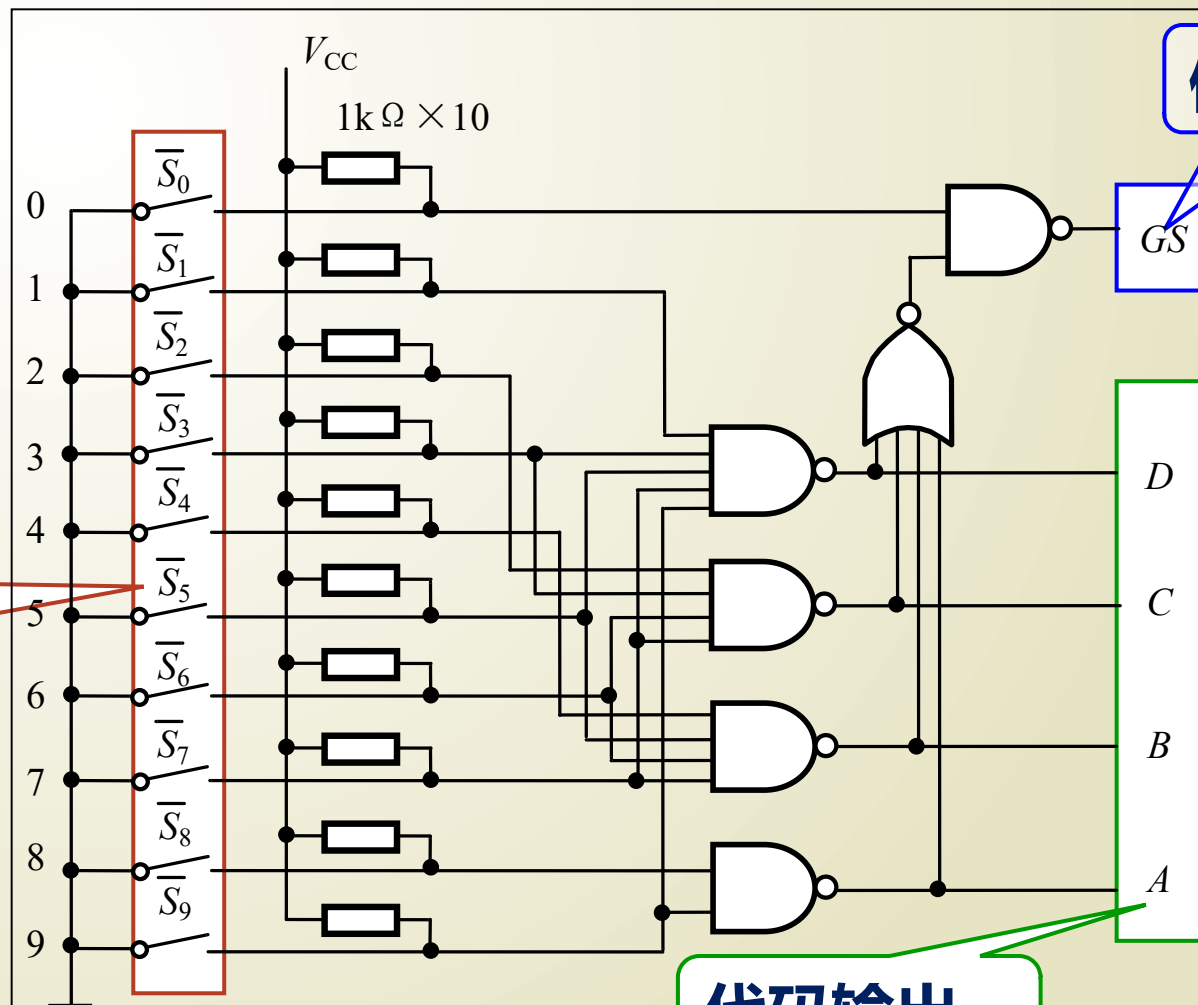
$I_1 = I_2 = 1$, $I_0 = I_1 = 0$ 时,

$Y_1 Y_0 = ?$ $Y_1 Y_0 = 11$

无法输出有效编码。

结论：普通编码器不能同时输入两个以上的有效编码信号

(2) 键盘输入 8421BCD 码编码器 (分析)



编码输入

使能标志

代码输出

键盘输入 8421BCD 码编码器功能表

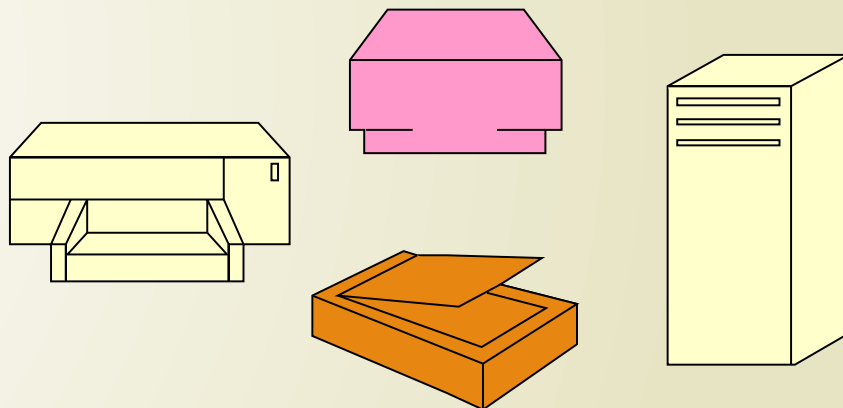
输 入										输 出				
$\overline{S_0}$	$\overline{S_1}$	$\overline{S_2}$	$\overline{S_3}$	$\overline{S_4}$	$\overline{S_5}$	$\overline{S_6}$	$\overline{S_7}$	$\overline{S_8}$	$\overline{S_9}$	A	B	C	D	GS
1	1	1	1	1	1	1	1	1	1	0	0	0	0	0
1	1	1	1	1	1	1	1	1	0	1	0	0	1	1
1	1	1	1	1	1	1	1	0	1	1	0	0	0	1
1	1	1	1	1	1	1	0	1	1	0	1	1	1	1
1	1	1	1	1	1	0	1	1	1	0	1	1	0	1
1	1	1	1	1	0	1	1	1	1	0	1	0	1	1
1	1	1	1	0	1	1	1	1	1	0	1	0	0	1
1	1	1	0	1	1	1	1	1	1	0	0	1	1	1
1	1	0	1	1	1	1	1	1	1	0	0	1	0	1
1	0	1	1	1	1	1	1	1	1	0	0	0	1	1
0	1	1	1	1	1	1	1	1	1	0	0	0	0	1

该编码器输入低电平有效，输出高电平有效，GS 为标志位。

3. 优先编码器

优先编码器的提出：

实际应用中，经常有两个或更多输入编码信号同时有效。



必须根据轻重缓急，规定好这些外设允许操作的先后次序，即优先级别。

识别多个编码请求信号的优先级别，并进行相应编码的逻辑部件称为优先编码器。

8-3 线优先编码器功能表

输 入									输 出				
EI	I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0	GS	EO
0	×	×	×	×	×	×	×	×	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	1
1	1	×	×	×	×	×	×	×	1	1	1	1	0
1	0	1	×	×	×	×	×	×	1	1	0	1	0
1	0	0	1	×	×	×	×	×	1	0	1	1	0
1	0	0	0	1	×	×	×	×	1	0	0	1	0
1	0	0	0	0	1	×	×	×	0	1	1	1	0
1	0	0	0	0	0	1	×	×	0	1	0	1	0
1	0	0	0	0	0	0	1	×	0	0	1	1	0
1	0	0	0	0	0	0	0	1	0	0	0	1	0

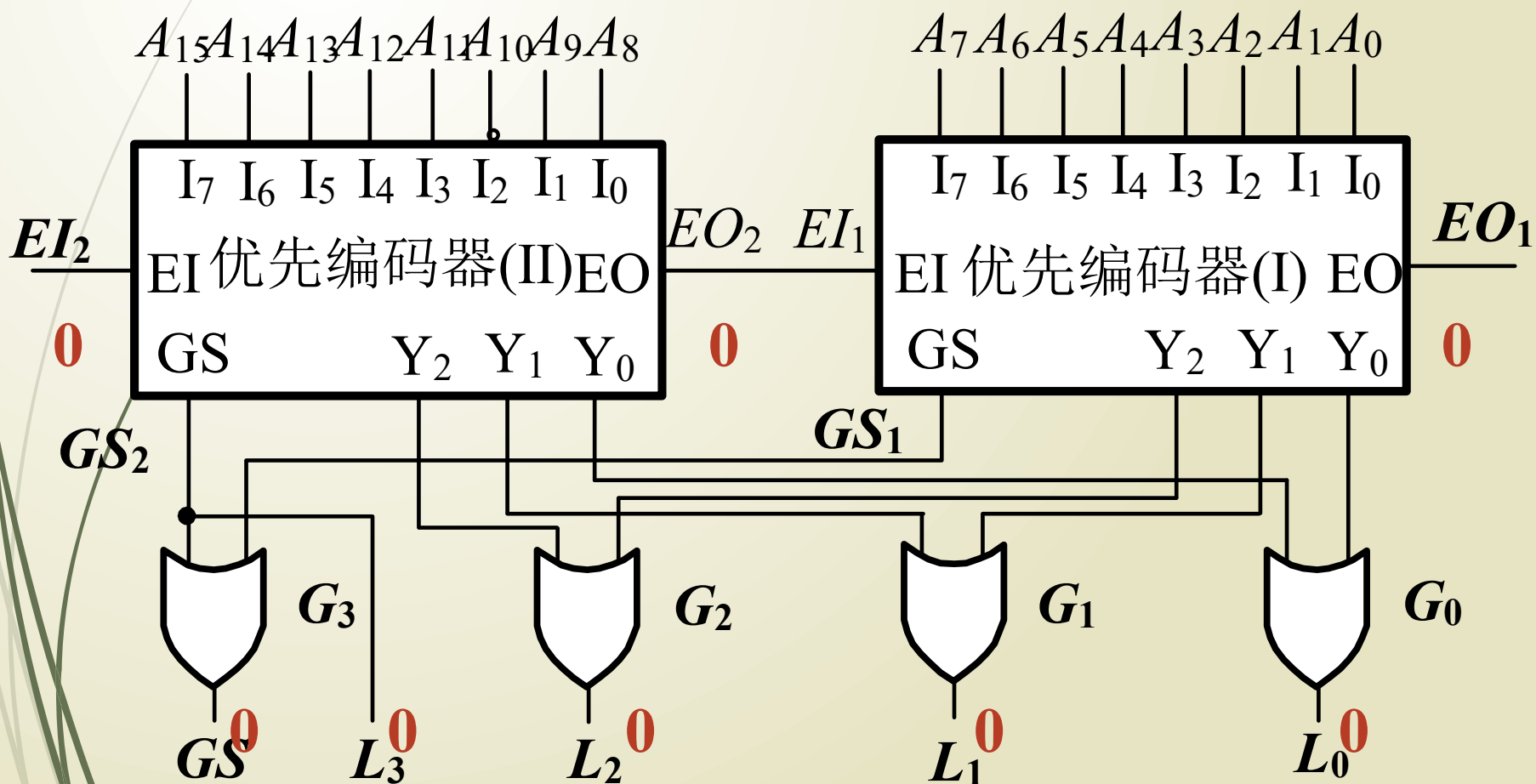
输入高电平有效，输出 $Y_2Y_1Y_0$ 为二进制代码

输入编码信号优先级从高到低为 $I_7 \sim I_0$

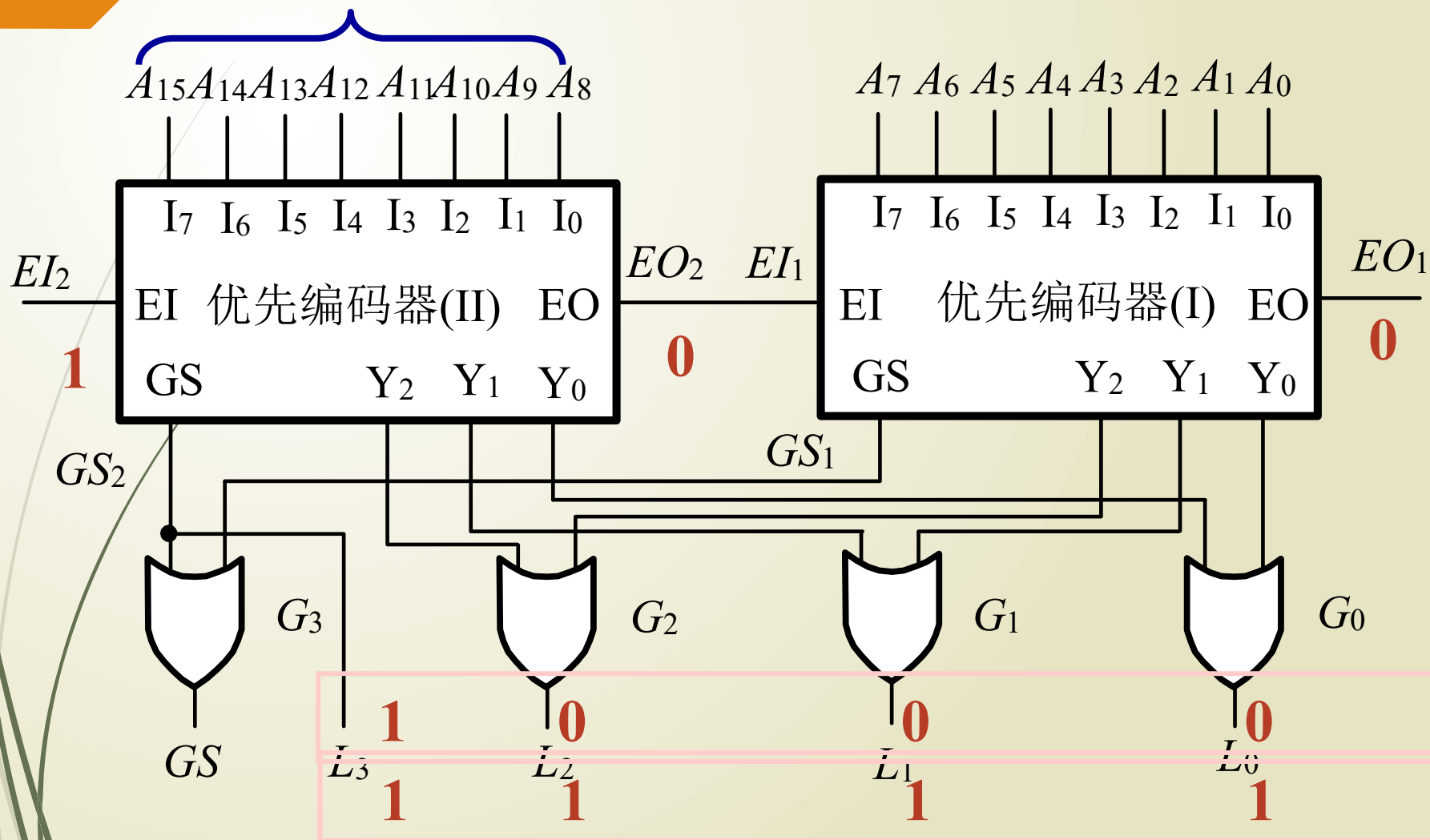
为什么要设计 GS、EO 输出信号？

用 2 个 8 线 - 3 线优先编码器构成 16 线 - 4 线优先编码器，
其逻辑图如下图所示，试分析其工作原理。

当使能端 $EI = 0$ 时，无编码输出。



若有效电平输入



4.4.2 译码器 / 数据分配器

1. 译码器的定义与分类

译码：译码是编码的逆过程，它可将二进制码翻译成代表某一特定含义的信号。（即电路的某种状态）

译码器：具有译码功能的逻辑电路称为译码器。

译码器的分类：

唯一地址译码器

将一系列代码转换成与之一一对应的有效信号。

常见的唯一地址译码器：

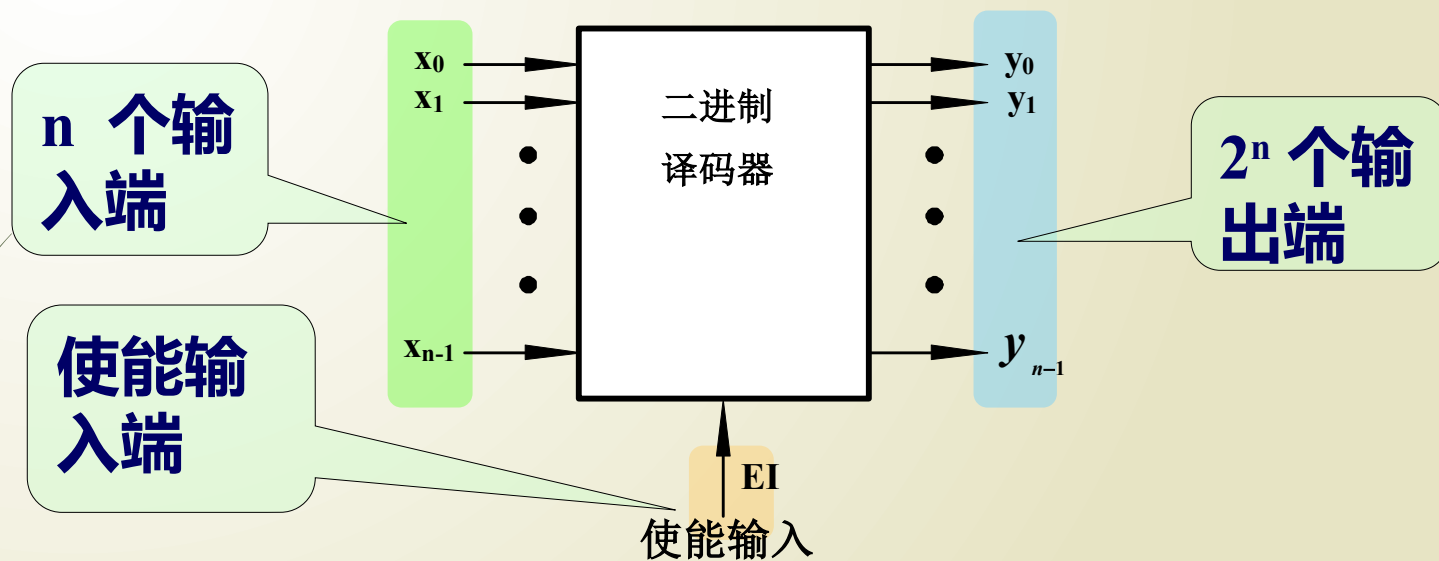
- 二进制译码器
- 二—十进制译码器
- 显示译码器

代码变换器

将一种代码转换成另一种代码。

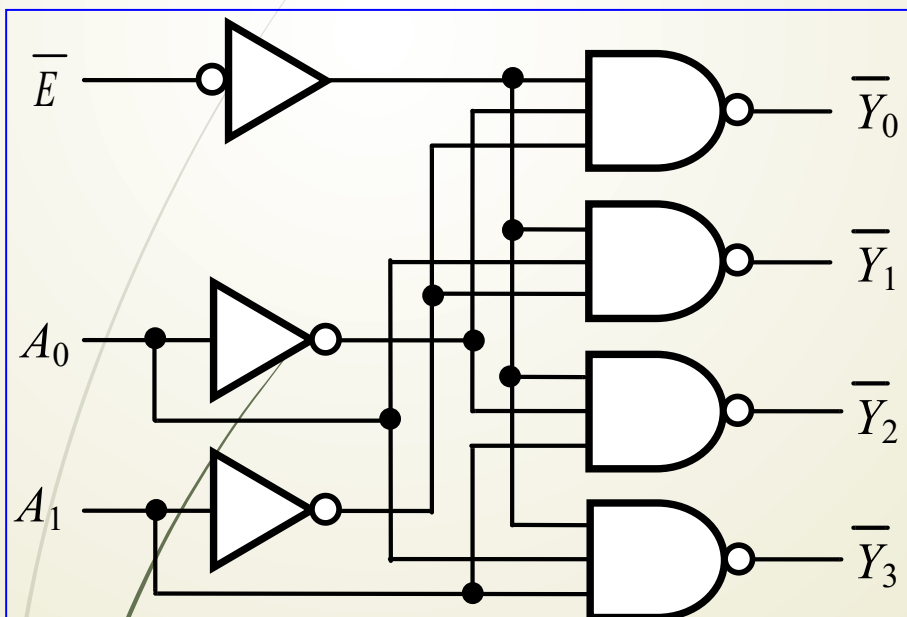
2. 译码器电路及应用

(1) 二进制译码器



设输入端的个数为 n , 输出端的个数为 M
则有 $M=2^n$

(a) 2线-4线译码器 (分析)



功能表

输入			输出			
$\overline{E}$	A_1	A_0	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$
1	×	×	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

$$\overline{Y}_0 = \overline{\overline{E} \overline{A_1} \overline{A_0}}$$

$$\overline{Y}_1 = \overline{\overline{E} \overline{A_1} A_0}$$

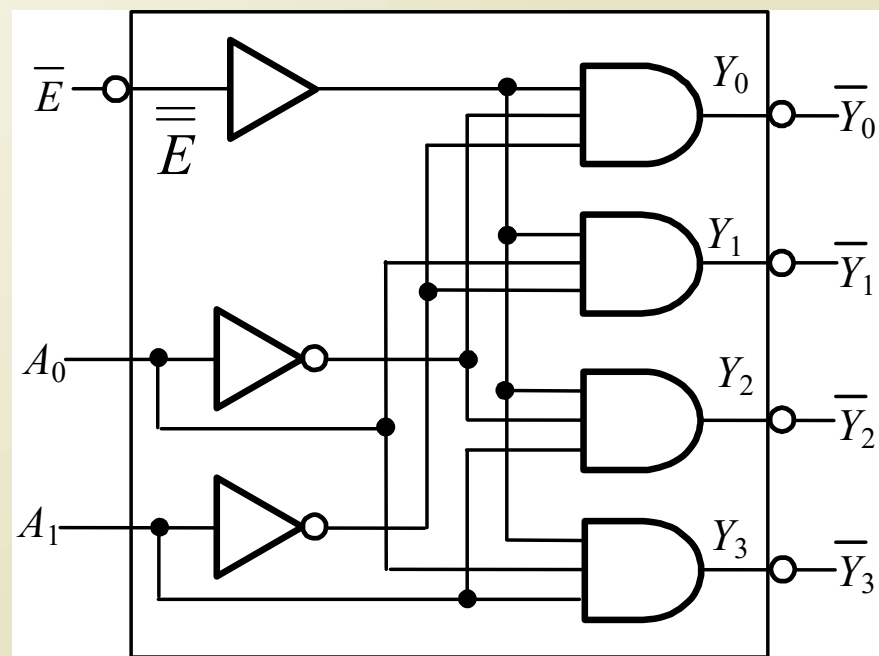
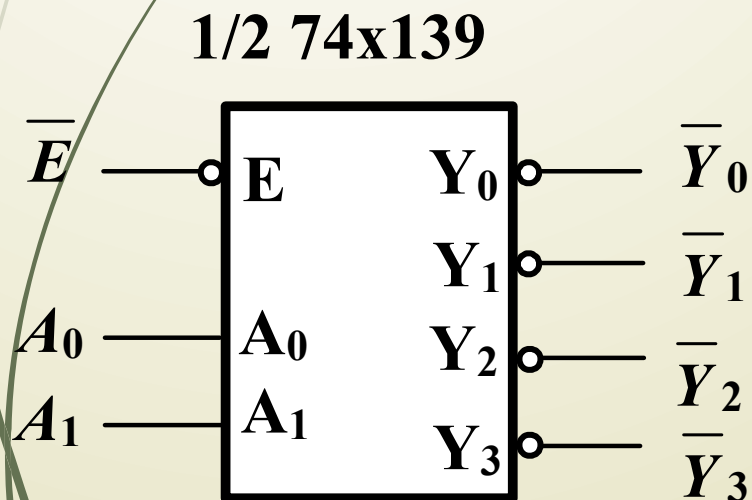
$$\overline{Y}_2 = \overline{\overline{E} A_1 \overline{A_0}}$$

$$\overline{Y}_3 = \overline{\overline{E} A_1 A_0}$$

(a) 2 线 -4 线译码器

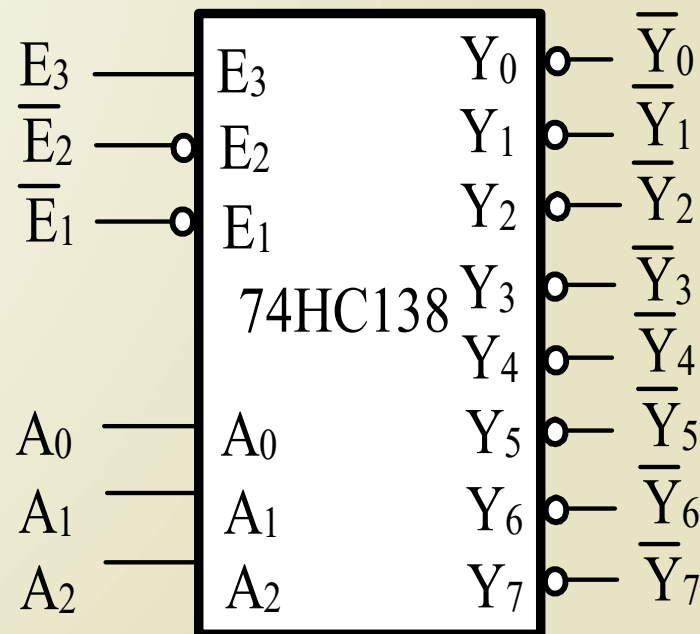
---- 逻辑符号说明

逻辑符号框外部的符号，表示外部输入或输出信号名称，字母上面的“—”号说明该输入或输出是低电平有效。符号框内部的输入、输出变量表示其内部的逻辑关系。在推导表达式的过程中，如果低有效的输入或输出变量（如）上面的“—”号参与运算（如 $\bar{E}$ 变为 E ），则在画逻辑图或验证真值表时，注意将其还原为低有效符号。



(b) 3 线 -8 线译码器 (74HC138)

- ◆ 使能输入 E_3 高有效, $\overline{E_2}$ 、 $\overline{E_1}$ 低有效。
- ◆ 输入 $A_2A_1A_0$ 高有效。
- ◆ 输出 $\overline{Y_7} \sim \overline{Y_0}$ 低有效。
- ◆ 低有效变量上面加 “ $\overline{\quad}$ ”。



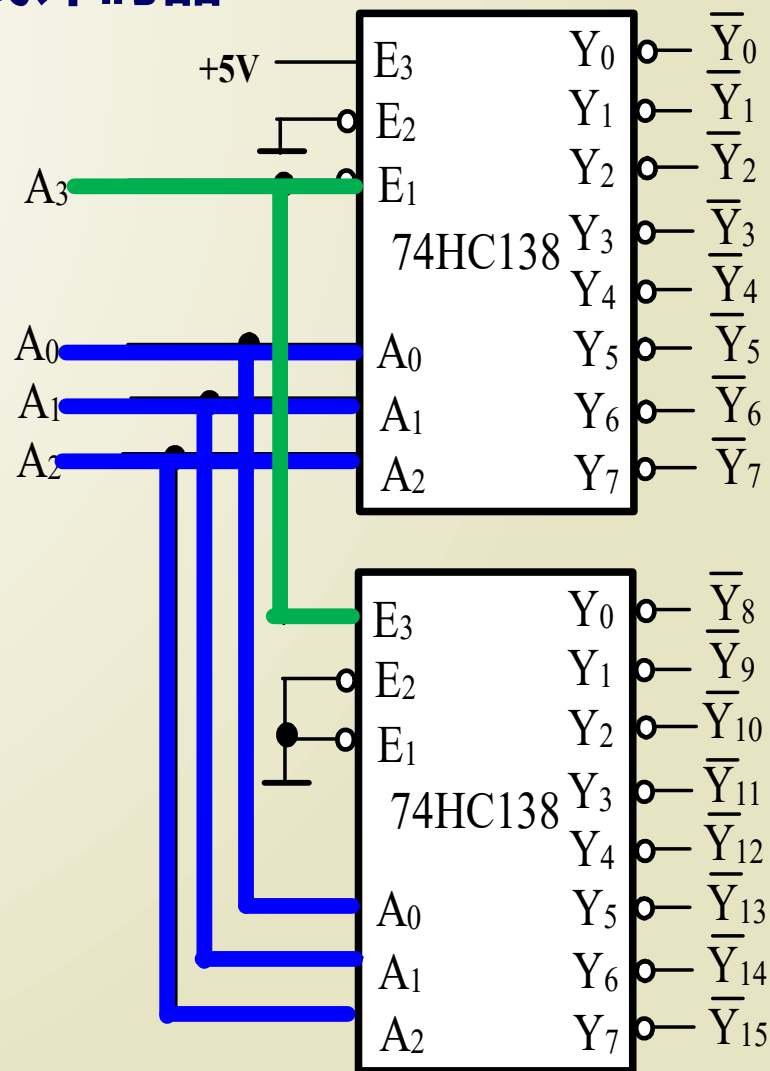
逻辑符号

3 线-8 线译码器 (74HC138) 功能表

[illegible]

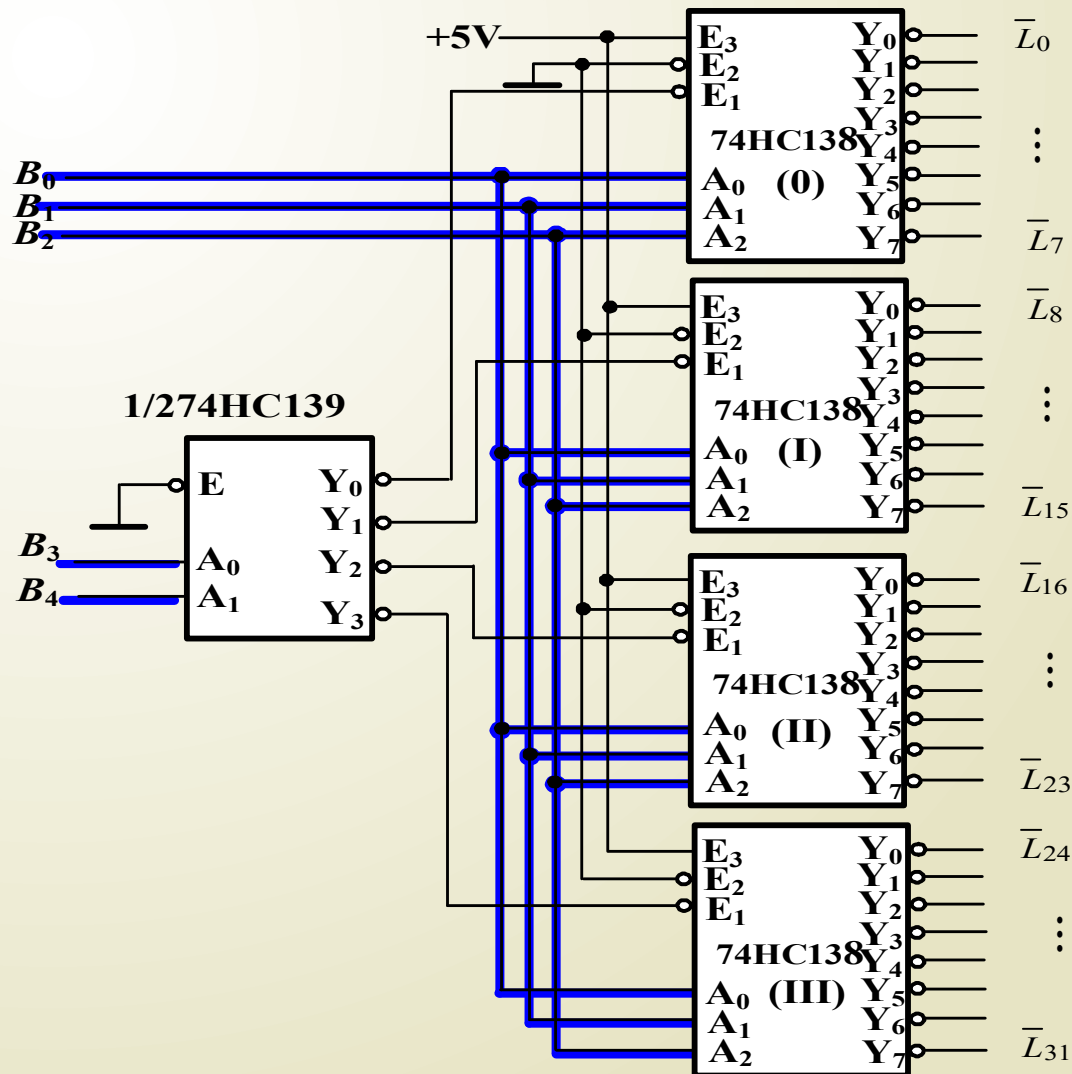
(c) 译码器的扩展

用 74X138 构成 4 线 -16 线译码器



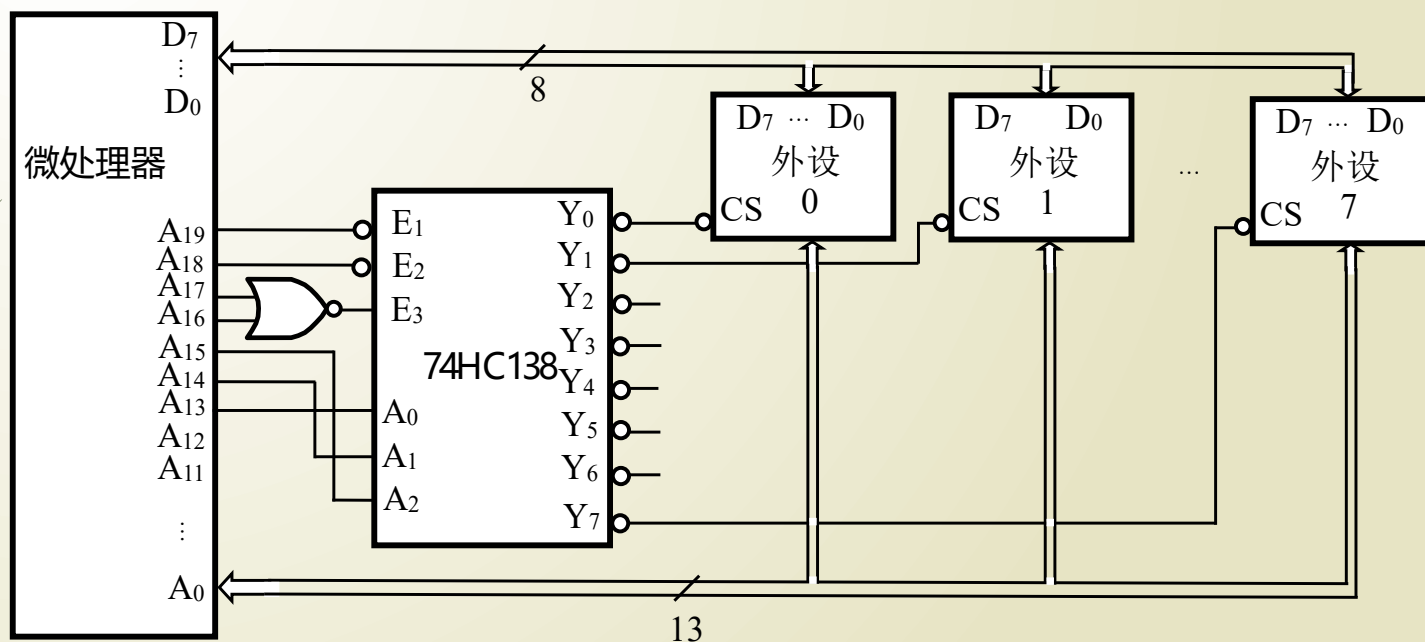
(c) 译码器的扩展

用 74X139 和 74X138 构成 5 线 -32 线译码器



(d) 译码器应用

译码器常用于识别不同设备。当 $A_{19}A_{18}A_{17}A_{16}=0000$, $A_{15}A_{14}A_{13}=000$, 选中外设 0。



$$\begin{aligned}\overline{CS}_0 &= \overline{Y}_0 = \overline{\overline{A}_{19} \overline{A}_{18} (A_{17} + A_{16}) \overline{A}_{15} \overline{A}_{14} \overline{A}_{13}} \\ &= A_{19} + A_{18} + A_{17} + A_{16} + A_{15} + A_{14} + A_{13}\end{aligned}$$

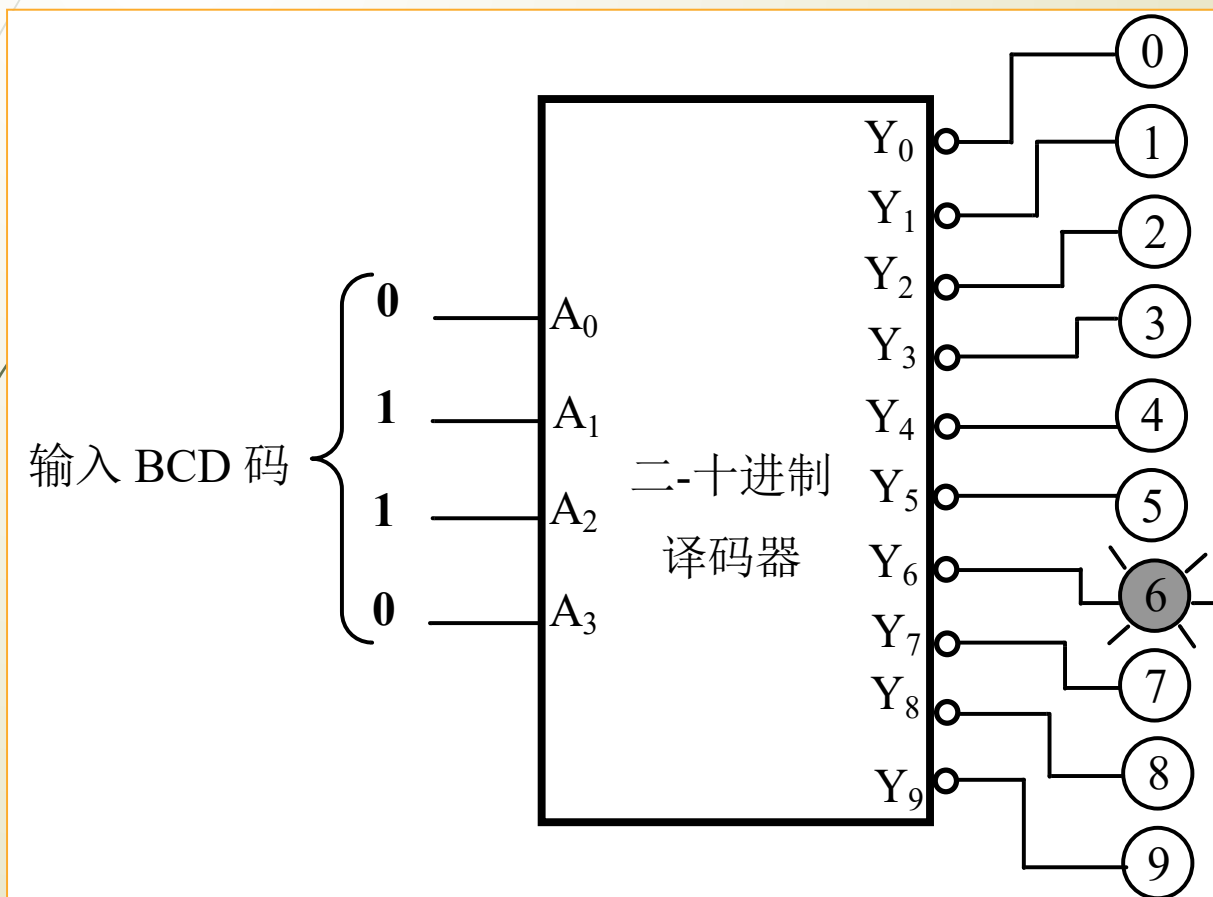
真值表

对于 BCD 代码以外的伪码 (1010 ~ 1111 这 6 个代码) $Y_0 \sim Y_9$ 均为高电平。

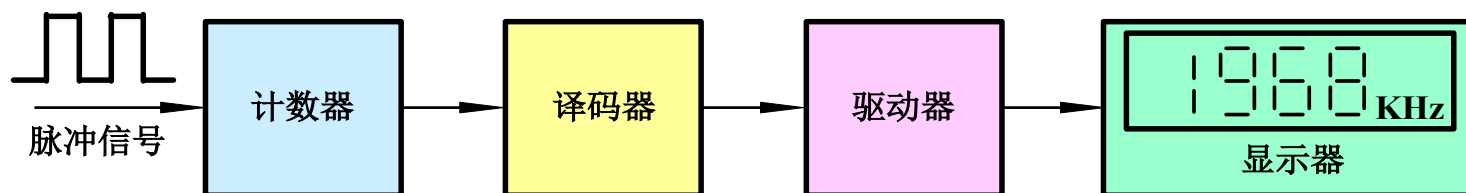
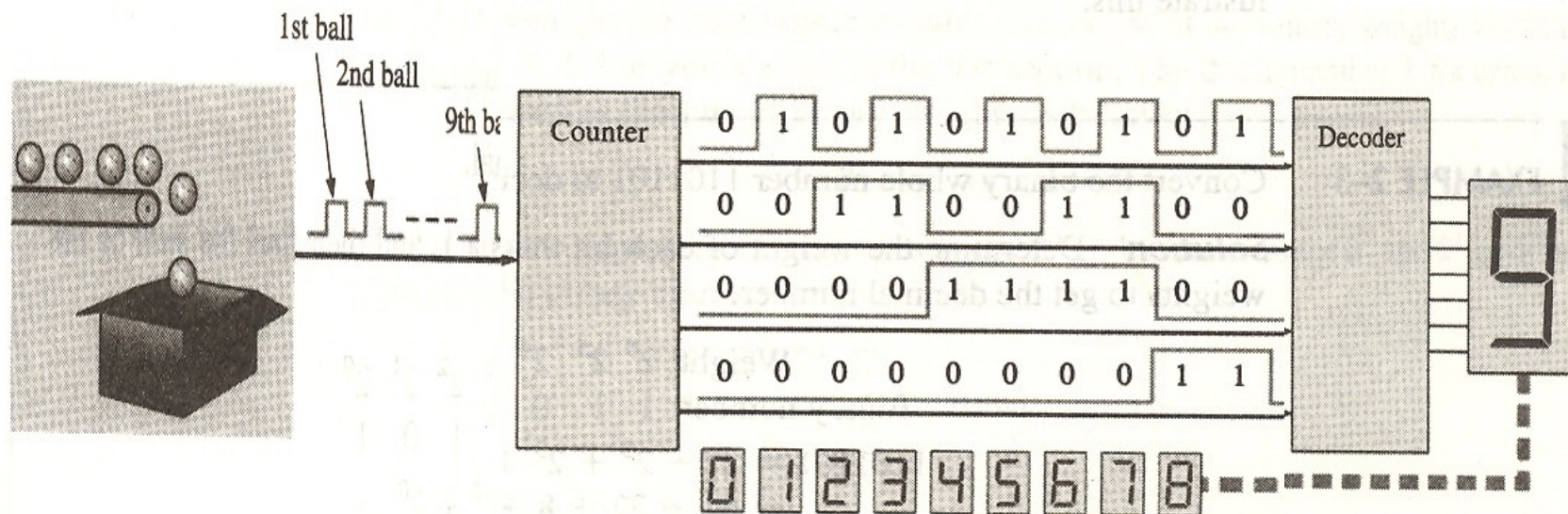
[illegible]

二-十进制译码器功能：

将 8421BCD 码译成为 10 个状态输出。

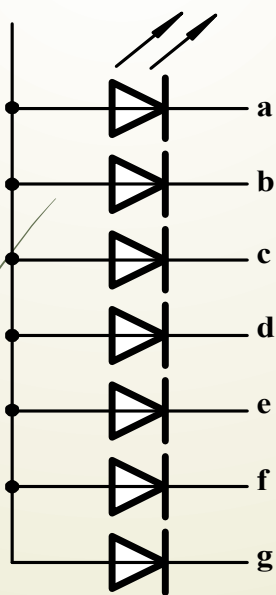


(3) 显示译码器

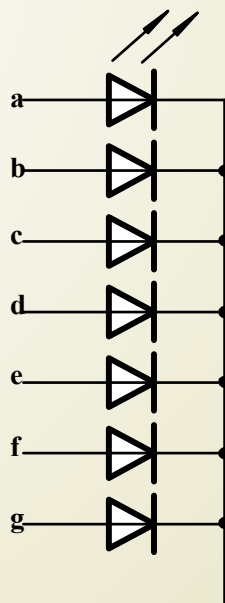


1. 七段显示译码器

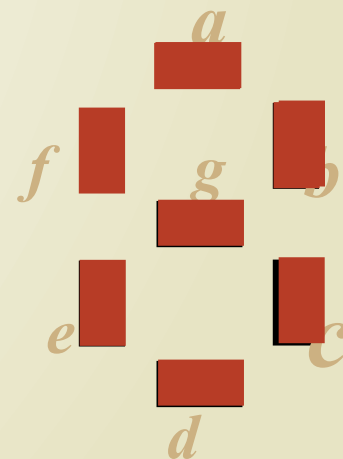
(1) 最常用的显示器有：半导体发光二极管和液晶显示器。



共阳极显示器



共阴极显示器

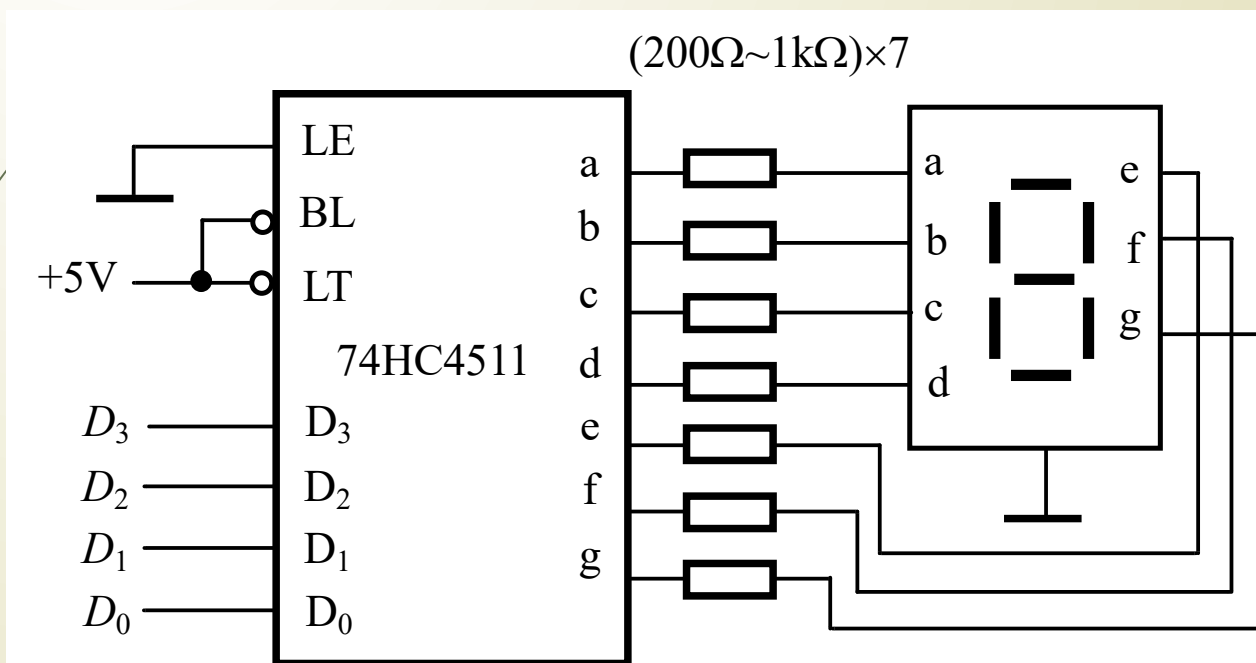


显示器分段布局图

常用的集成七段显示译码器

-----CMOS 七段显示译码器 74HC4511

显示译码器与显示器的连接方式



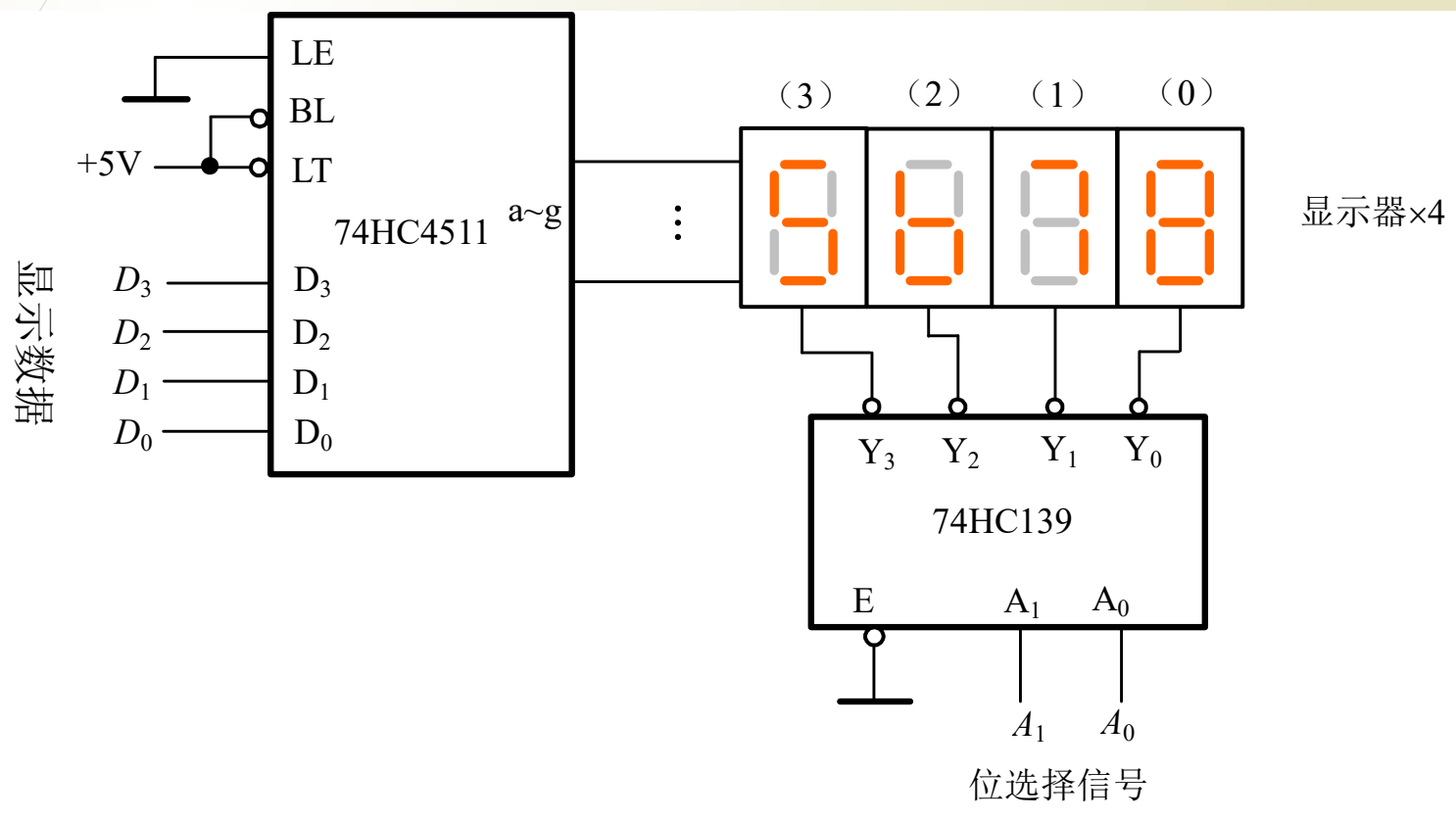
CMOS 七段显示译码器 74HC4511 功能表

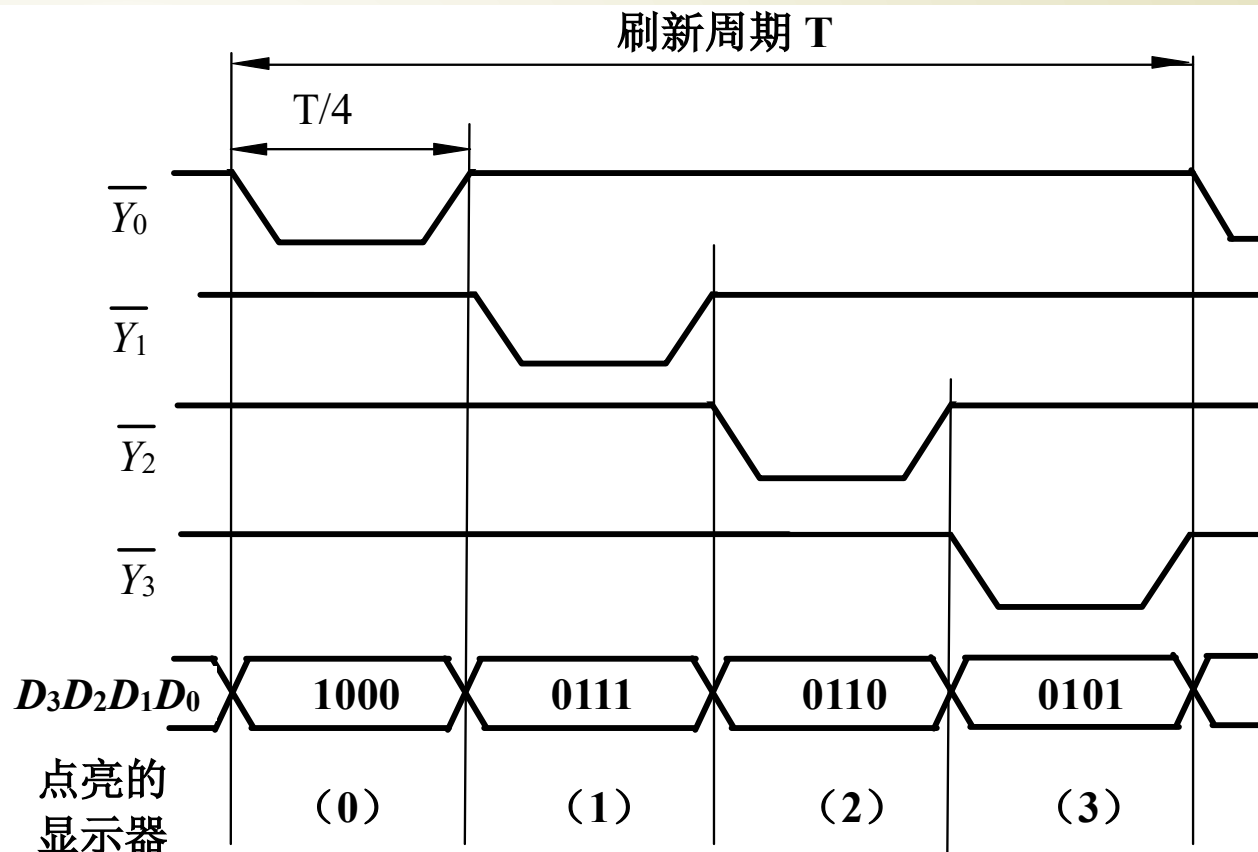
十进制或功能	输 入							输 出							字形
	LE	$\overline{BL}$	$\overline{LT}$	D_3	D_2	D_1	D_0	a	b	c	d	e	f	g	
0	0	1	1	0	0	0	0	1	1	1	1	1	1	0	0
1	0	1	1	0	0	0	1	0	1	1	0	0	0	0	1
2	0	1	1	0	0	1	0	1	1	0	1	1	0	1	2
3	0	1	1	0	0	1	1	1	1	1	1	0	0	1	3
4	0	1	1	0	1	0	0	0	1	1	0	0	1	1	4
5	0	1	1	0	1	0	1	1	0	1	1	0	1	1	5
6	0	1	1	0	1	1	0	0	0	1	1	1	1	1	6
7	0	1	1	0	1	1	1	1	1	1	0	0	0	0	7
8	0	1	1	1	0	0	0	1	1	1	1	1	1	1	8
9	0	1	1	1	0	0	1	1	1	1	1	0	1	1	9

CMOS 七段显示译码器 74HC4511 功能表 (续)

十进制 或功能	输 入							输 出							字形
	LE	$\overline{BL}$	$\overline{LT}$	D_3	D_2	D_1	D_0	a	b	c	d	e	f	g	
10	0	1	1	1	0	1	0	0	0	0	0	0	0	0	熄灭
11	0	1	1	1	0	1	1	0	0	0	0	0	0	0	熄灭
12	0	1	1	1	1	0	0	0	0	0	0	0	0	0	熄灭
13	0	1	1	1	1	0	1	0	0	0	0	0	0	0	熄灭
14	0	1	1	1	1	1	0	0	0	0	0	0	0	0	熄灭
15	0	1	1	1	1	1	1	0	0	0	0	0	0	0	熄灭
灯 测 试	×	×	0	×	×	×	×	1	1	1	1	1	1	1	8
灭 灯	×	0	1	×	×	×	×	0	0	0	0	0	0	0	熄灭
锁 存	1	1	1	×	×	×	×	*							*

例 由译码器、显示译码及 4 个七段显示器构成的 4 位动态显示电路如图所示，试分析工作原理。



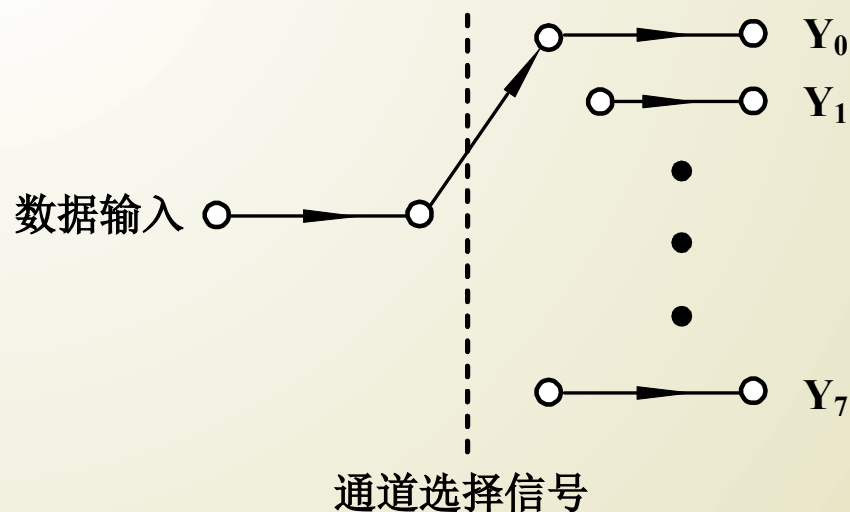


位选择信号 A_1 、 A_0 控制 $\overline{Y_3} \sim \overline{Y_0}$ 依次产生低电平，使 4 个显示器轮流显示。要显示的数据组依次送到 $D_3D_2D_1D_0$ 分别在 4 个显示器上显示。利用人的视觉暂留时间，当刷新频率达到一定数值（如 40Hz，即 $T=1/40\text{Hz}=0.025\text{s}$ ）可看到稳定的数字。

3. 数据分配器

——用 74HC138 组成数据分配器

数据分配器示意图

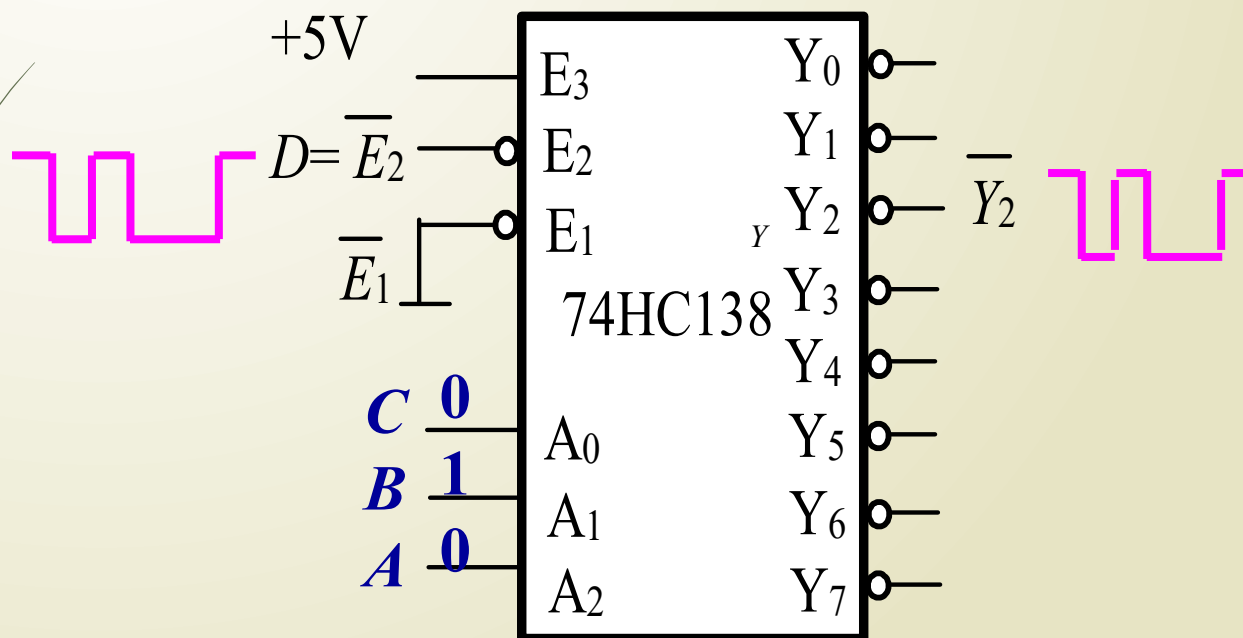


数据分配器：相当于多输出的单刀多掷开关，是将公共数据线上的数据按需要送到不同的通道上去的逻辑电路。

用译码器实现数据分配器

$$\overline{Y_2} = \overline{E_3} \overline{E_2} D \overline{A} \overline{B} \overline{C}$$

当 $ABC = 010$ 时, $\overline{Y_2} = D$



[illegible]

例：试用门电路设计一个具有低电平使能控制的 1 线—4 线数据分配器，使能信号无效时，电路所有的输出为高阻态。当通道选择信号将 1 路输入信号连接到其中 1 路输出端时，其他输出端为高阻状态。

1. 列真值表

输出端有 3 种状态（0、1、z），输出级是 4 个三态门组成。其控制信号由 E、S1、S0 共同作用产生。

输 入	三 态 门 控 制 信 号						输 出			
三态门控制信号 S ₁ S ₀ C ₃ C ₂ C ₁ C ₀	S ₁	S ₀	C ₃	C ₂	C ₁	C ₀	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	0	1	z	z	z	In
0	0	1	0	0	1	0	z	z	In	z
0	1	0	0	1	0	0	z	In	z	z
0	1	1	1	0	0	0	In	z	z	z
1	x	x	0	0	0	0	z	z	z	z

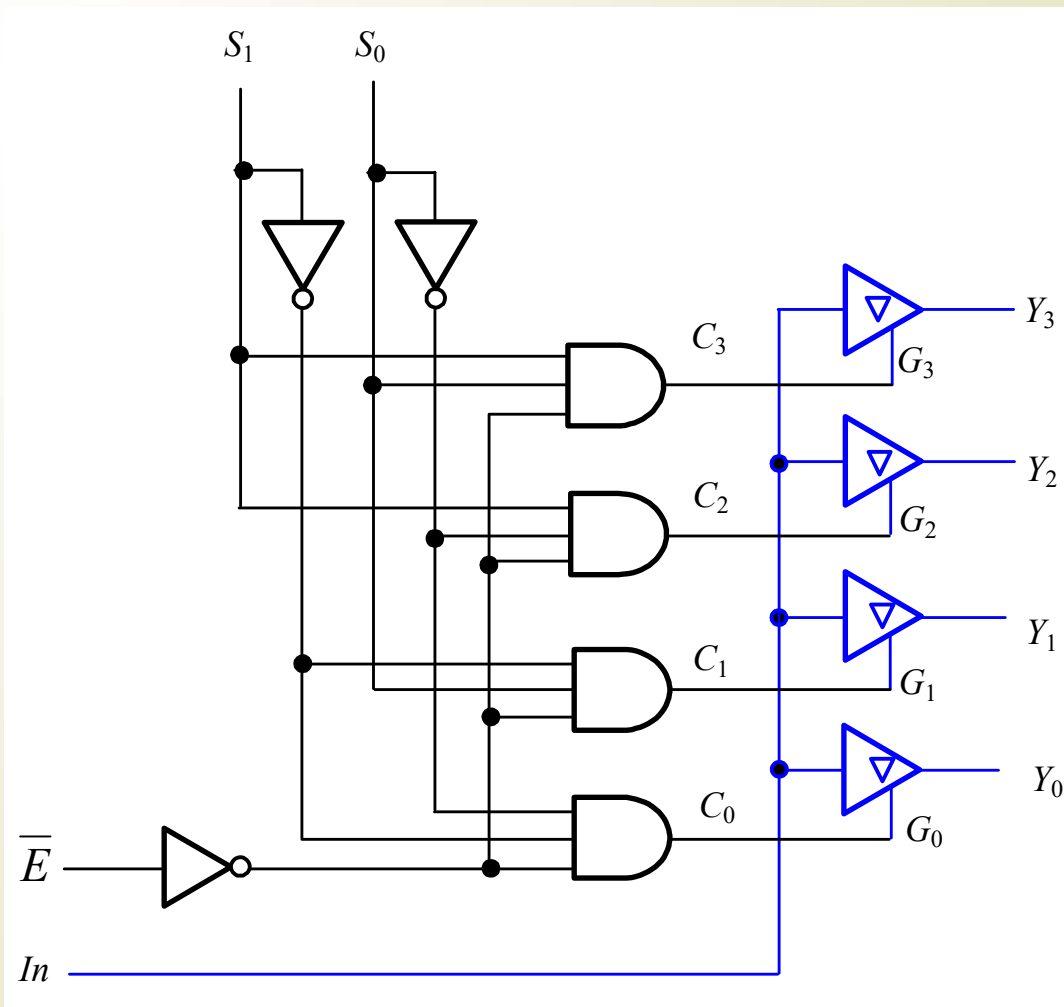
2. 写出 4 个三态门控制端的逻辑表达式

$$C_0 = \overline{\overline{E}} \cdot \overline{S_1} \cdot \overline{S_0} \quad C_1 = \overline{\overline{E}} \cdot \overline{S_1} \cdot S_0$$

$$C_2 = \overline{\overline{E}} \cdot S_1 \cdot \overline{S_0} \quad C_3 = \overline{\overline{E}} \cdot S_1 \cdot S_0$$

输 入		三 态 门 控 制 信 号					输 出			
	S_1	S_0	C_3	C_2	C_1	C_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	1	z	z	z	In
0	0	1	0	0	1	0	z	z	In	z
0	1	0	0	1	0	0	z	In	z	z
0	1	1	1	0	0	0	In	z	z	z
1	x	x	0	0	0	0	z	z	z	z

3. 画逻辑电路

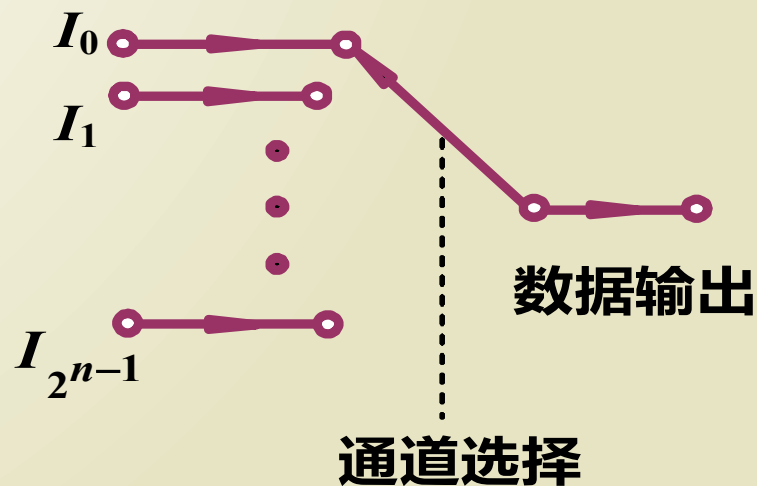


4.4.3 数据选择器

1、数据选择器的定义与功能

数据选择器： 能够实现数据选择功能的逻辑电路。它的作用相当于多个输入的单刀多掷开关， 又称“多路开关”。

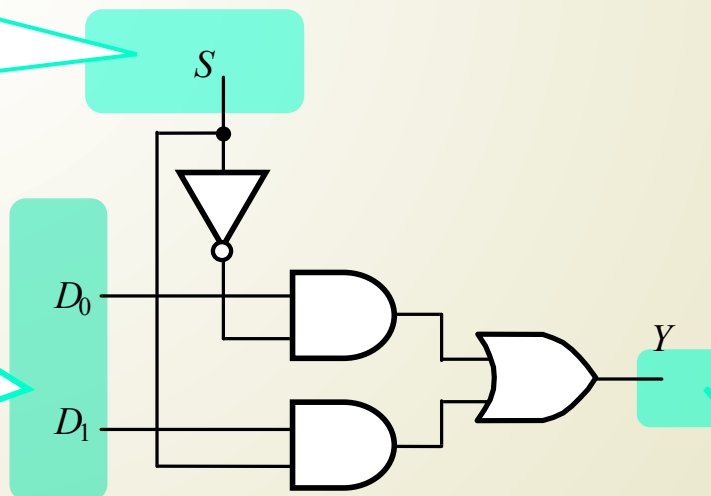
数据选择的功能： 在通道选择信号的作用下， 将多个通道的数据分时传送到公共的数据通道上去的。



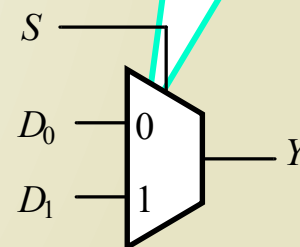
2 选 1 数据选择器

1 位地址
码输入端

数据
输入端



逻辑符号



1 路数据
输出端

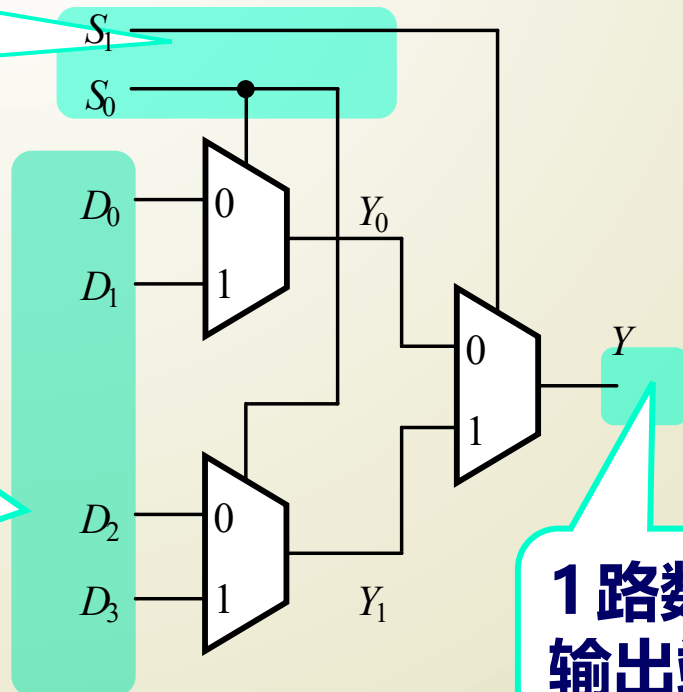
4 选 1 数据选择器

(1) 逻辑电路

由 3 个 2 选 1 数据选择器构成 4 选 1 数据选择器。

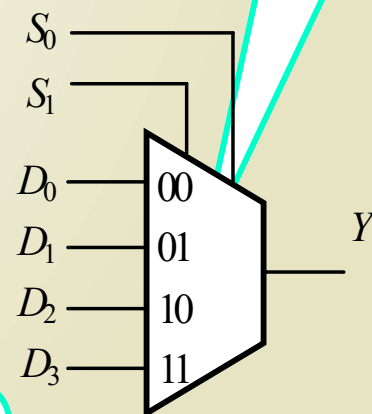
2 位地址
码输入端

数据
输入端



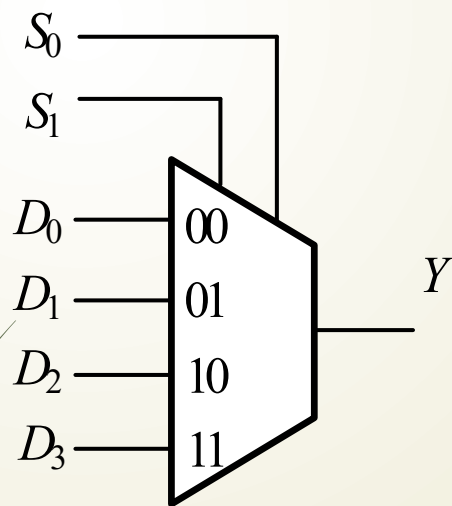
1 路数据
输出端

逻辑符号



(2) 工作原理及逻辑功能

真值表



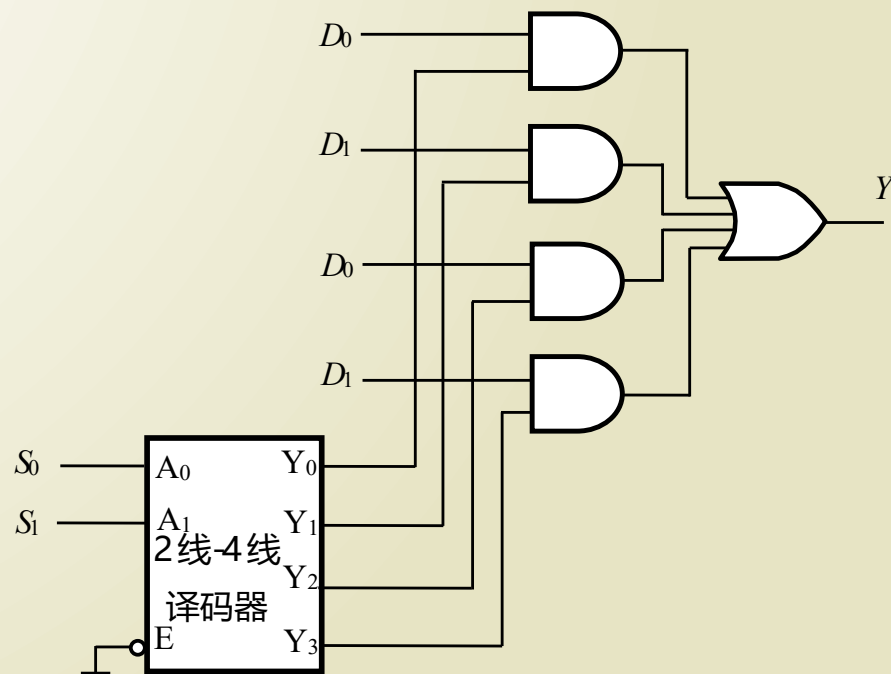
选择输入		输 出
S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

$$Y = \overline{S_1}\overline{S_0}D_0 + \overline{S_1}S_0D_1 + S_1\overline{S_0}D_2 + S_1S_0D_3$$

$$Y = D_0m_0 + D_1m_1 + D_2m_2 + D_3m_3$$

(3) 用译码器构成数据选择器

译码器与数据选择器都具有选择功能，因此可以用二进制译码器构成数据选择器。用 S_1S_0 选择 D_0 、 D_1 、 D_2 或 D_3 中的一个数据传送到输出端。



(4) 数据选择器实现逻辑函数

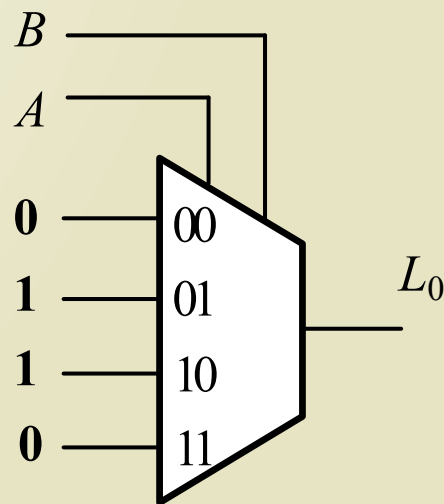
例 4.4.8 试用数据选择器实现下列逻辑函数

① 用 4 选 1 数据选择器实现 $L_0 = \overline{A}B + A\overline{B}$

② 用 2 选 1 数据选择器和必要的逻辑门实现 $L_1 = \overline{A}\overline{C} + BC$

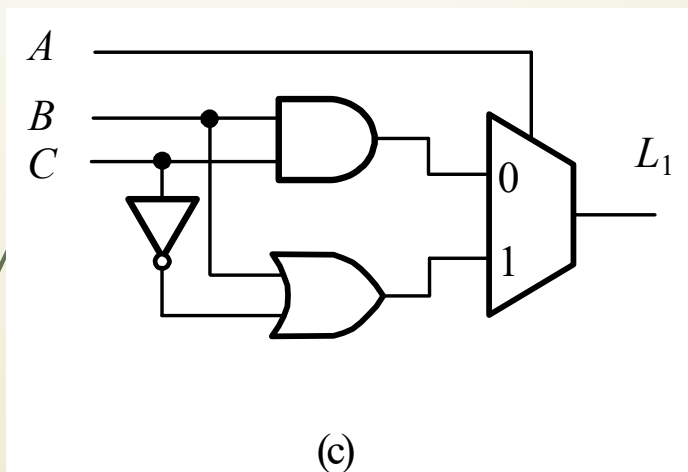
$$Y = \overline{S_1}\overline{S_0}D_0 + \overline{S_1}S_0D_1 + S_1\overline{S_0}D_2 + S_1S_0D_3$$

① 当 $S_1 = A, S_2 = B,$
 $D_0 = D_3 = 0, D_1 = D_2 = 1$



② 用 2 选 1 数据选择器和必要的逻辑门实现 $L_1 = \overline{A}C + BC$

(a) 从真值表考察 A 、 B 、 C 与 L_1 的关系。2 选 1 数据选择器只有 1 个选通端接输入 A ，表达式有 3 个变量。因此数据端需要输入 2 个变量。



输 入			输 出	
A	B	C	L_1	
0	0	0	0	$L_1 = BC$
0	0	1	0	
0	1	0	0	
0	1	1	1	
1	0	0	1	$L_1 = B + \overline{C}$
1	0	1	0	
1	1	0	1	
1	1	1	1	

② 用 2 选 1 数据选择器和必要的逻辑门实现 $L_1 = \overline{A}\overline{C} + BC$

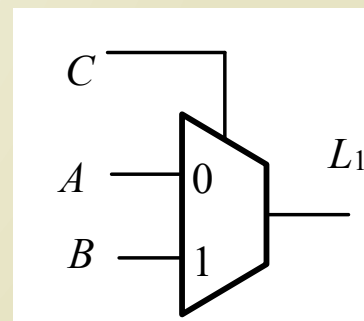
(b) 用香农展开定理: $f(X_1, X_2, \dots, X_n) = \overline{X_1} \cdot f(0, X_2, \dots, X_n) + X_1 \cdot f(1, X_2, \dots, X_n)$

对 A 展开, 结果与真值表的相同。

$$\begin{aligned} L_1 &= \overline{A}(0 \cdot \overline{C} + BC) + A(1 \cdot \overline{C} + BC) \\ &= \overline{A}BC + A(\overline{C} + BC) \\ &= \overline{A}BC + A(B + \overline{C}) \end{aligned}$$

对 C 展开, 成本更低。

$$\begin{aligned} L_1 &= \overline{C}(A \cdot 0 + B \cdot 0) + C(A \cdot 1 + B \cdot 1) \\ &= \overline{C}(A) + C(B) \end{aligned}$$



总结：

利用数据选择器实现函数的一般步骤：（变量数 = 选通端数）

a、将函数变换成最小项表达式

b、地址信号 S_2 、 S_1 、 S_0 作为函数的输入变量

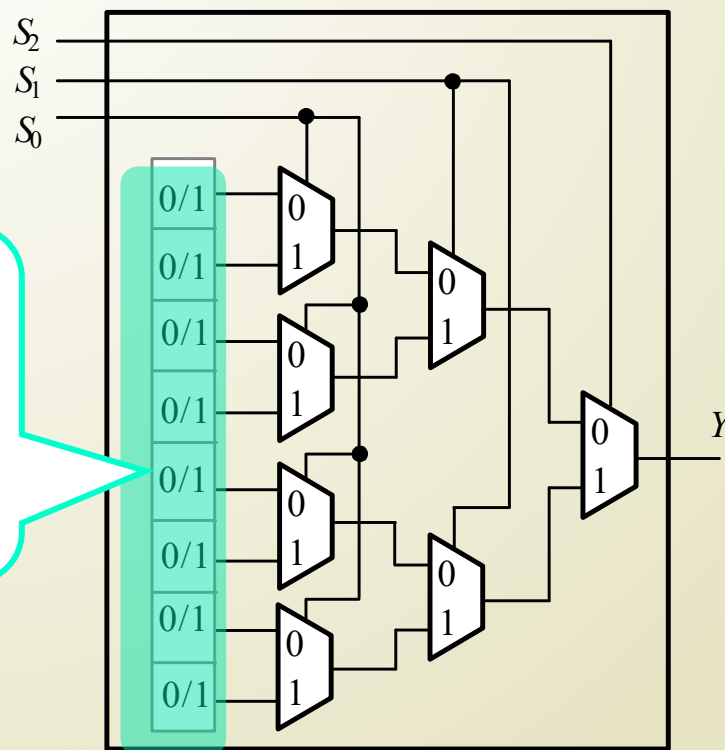
c、处理数据输入 $D_0 \sim D_7$ 信号电平。逻辑表达式中有 m_i ，则相应 $D_i = 1$ ，其他的数据输入端均为 0。

当变量数 $\square$ 选通端数，考虑如何将某些变量接入数据端。

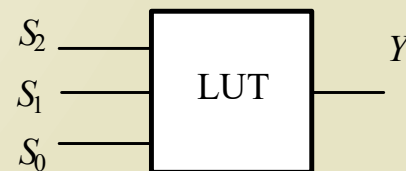
(5) 数据选择器构成查找表 LUT

构成 FPGA 基本单元的逻辑块主要是查找表 LUT。LUT 实质是一个小规模的存储器，以真值表的形式实现给定的逻辑函数。3 输入 LUT 的结构及逻辑符号如图。

存放 0
或 1 的
存储单元



(a) 结构



(b) 符号

用查找表 LUT 实现逻辑函数

$$L_1 = A\bar{C} + BC$$

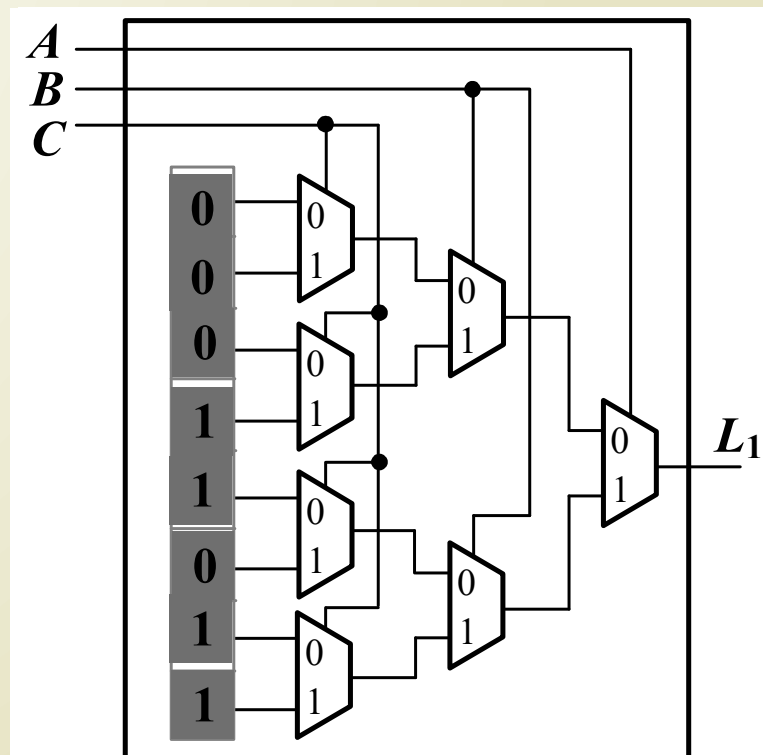
用 LUT 实现逻辑函数，变量 A、B、C 接选择输入端，对存储单元进行编程。

根据前面例题已知

$$\begin{aligned} L_1 &= A\bar{C} + BC \\ &= \sum m(3, 4, 6, 7) \end{aligned}$$

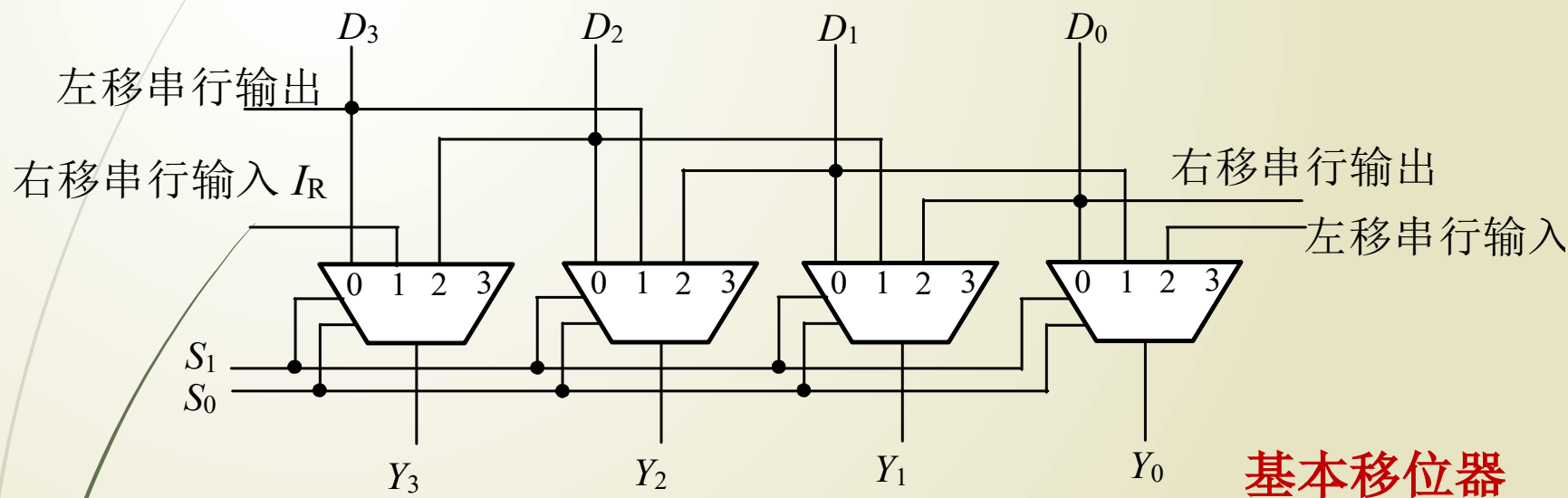
$$D_0 = D_1 = D_2 = D_5 = 0$$

$$D_3 = D_4 = D_6 = D_7 = 1$$



(6) 数据选择器构成移位器

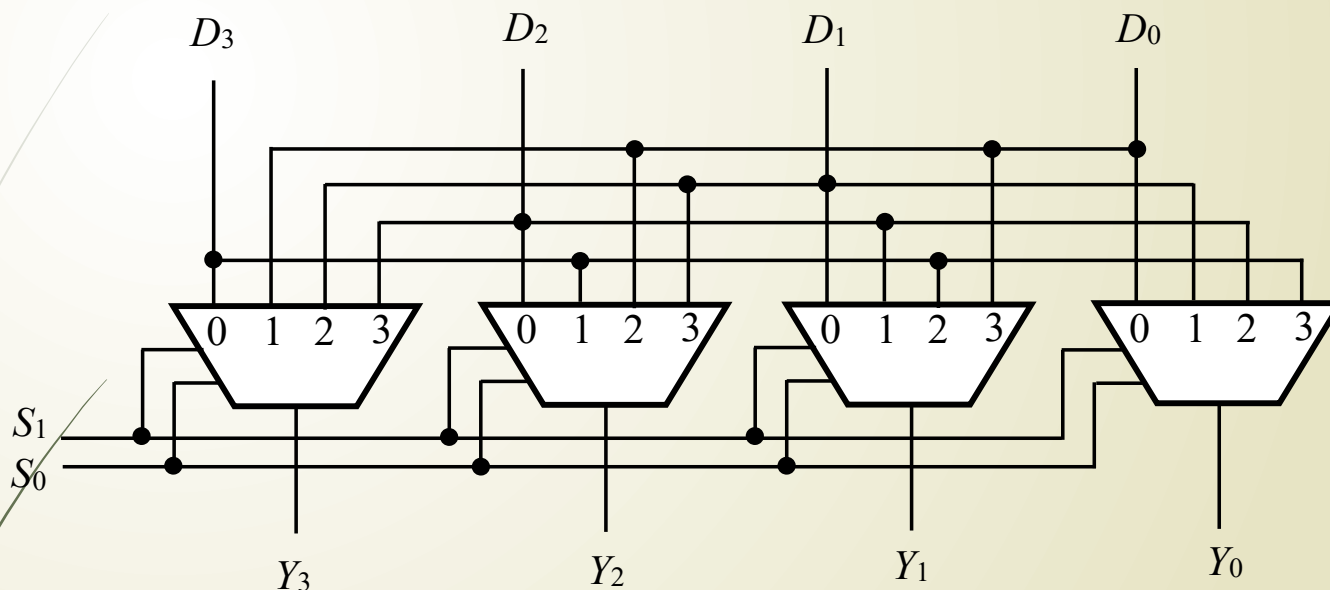
第6章介绍的移位寄存器的移位是在时钟脉冲控制下进行的，一个时钟脉冲只能移动一位。但在运算电路中常需要高速移位，这时便可采用组合逻辑电路构成的移位器。



选 择	输 出	功 能
$S_1 S_0$	$Y_3 Y_2 Y_1 Y_0$	
0 0	$D_3 D_2 D_1 D_0$	直接输出
0 1	$I_R D_3 D_2 D_1$	右移一位
1 0	$D_2 D_1 D_0 I_L$	左移一位

(6) 数据选择器构成移位器

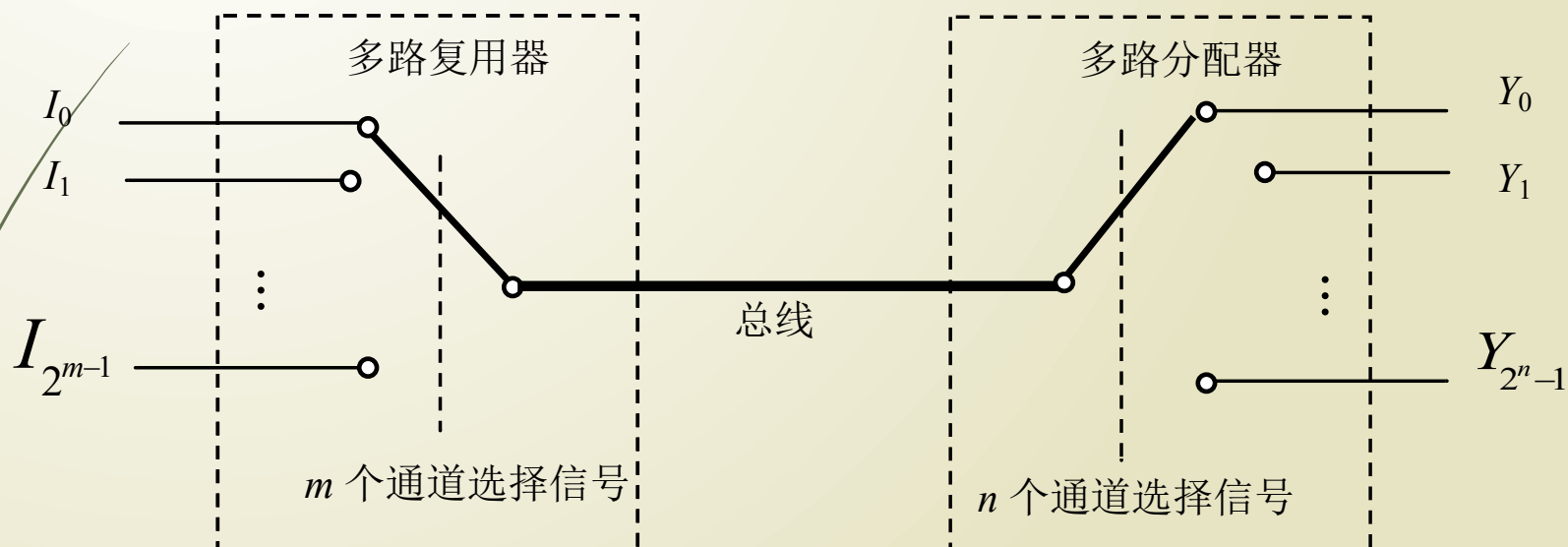
桶形移位器



选 择	输 出	功 能
$S_1 S_0$	$Y_3 Y_2 Y_1 Y_0$	
0 0	$D_3 D_2 D_1 D_0$	直接输出
0 1	$D_0 D_3 D_2 D_1$	循环右移一位（循环左移三位）
1 0	$D_1 D_0 D_3 D_2$	循环右移二位（循环左移二位）
1 1	$D_2 D_1 D_0 D_3$	循环右移三位（循环左移一位）

(7) 数据选择器、数据分配器与总线的连接

这种信息传输的基本原理在通信系统、计算机网络系统、以及计算机内部各功能部件之间的信息转送等等都有广泛的应用。



4.4.4 数值比较器

数值比较器：对两个 1 位数字进行比较（ A 、 B ），以判断其大小的逻辑电路。

1、1 位数值比较器（设计）

输入：两个一位二进制数 A 、 B 。

输出： $F_{A>B}=1$ ，表示 A 大于

$F_{A<B}=1$ ，表示 A 小于 B

$F_{A=B}=1$ ，表示 A 等于 B

1 位数值比较器

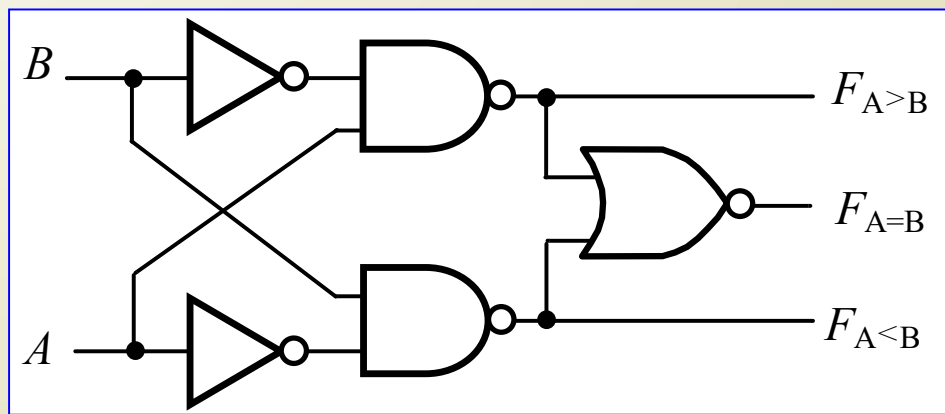
一位数值比较器真值表

输 入		输 出		
A	B	$F_{A>B}$	$F_{A<B}$	$F_{A=B}$
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

$$F_{A>B} = A \overline{B}$$

$$F_{A<B} = \overline{A} B$$

$$F_{A=B} = \overline{A} \overline{B} + AB$$



2、2 位数值比较器：

比较两个 2 位二进制数的大小的电

路

输入：两个 2 位二进制数 $A=A_1A_0$ 、 $B=B_1B_0$

能否用 1 位数值比较器设计两位数值比较器？

用一位数值比较器设计多位数值比较器的原则

当高位（ A_1 、 B_1 ）不相等时，无需比较低位（ A_0 、 B_0 ），高位比较的结果就是两个数的比较结果。

当高位相等时，两数的比较结果由低位比较的结果决定。

真值表

输 入				输 出		
A_1	B_1	A_0	B_0	$F_{A>B}$	$F_{A<B}$	$F_{A=B}$
$A_1 > B_1$		×		1	0	0
$A_1 < B_1$		×		0	1	0
$A_1 = B_1$		$A_0 > B_0$		1	0	0
$A_1 = B_1$		$A_0 < B_0$		0	1	0
$A_1 = B_1$		$A_0 = B_0$		0	0	1

$$F_{A>B} = (A_1 > B_1) + (A_1 = B_1)(A_0 > B_0)$$

$$F_{A<B} = (A_1 < B_1) + (A_1 = B_1)(A_0 < B_0)$$

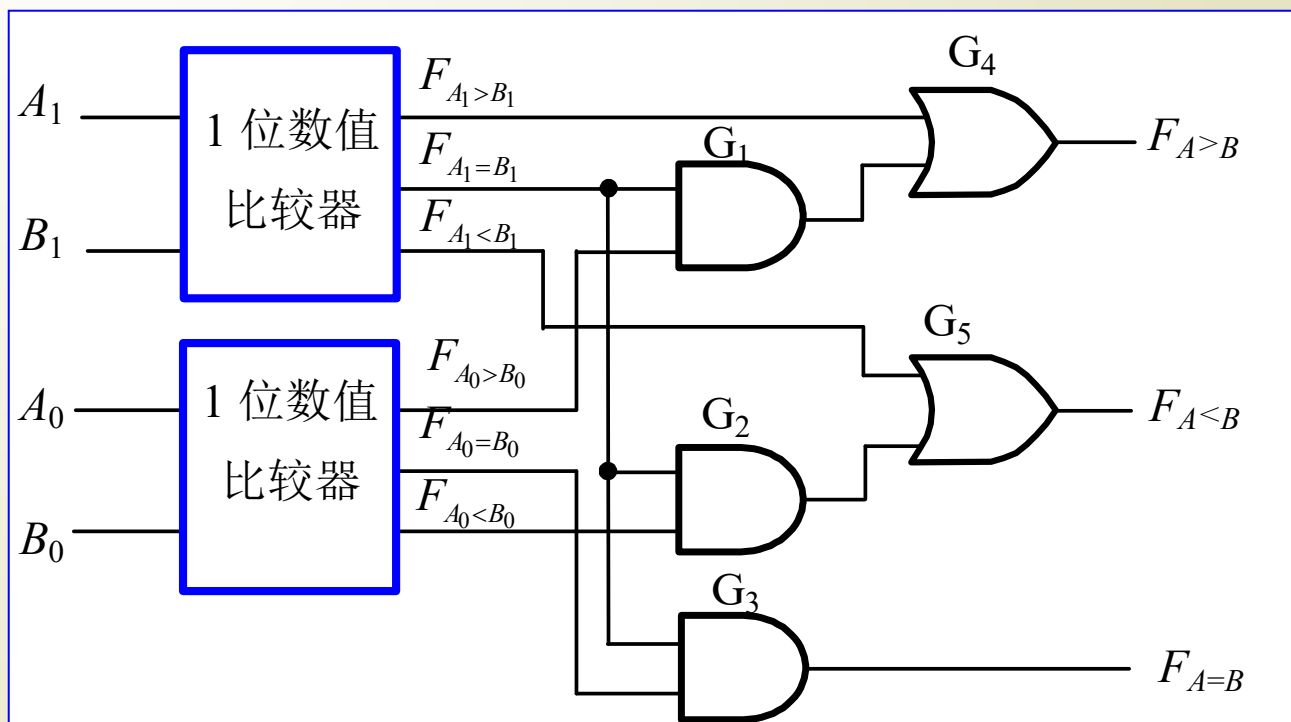
$$F_{A=B} = (A_1 = B_1)(A_0 = B_0)$$

注意：上述不是真正的逻辑函数表达式，只示意逻辑关系。

$$F_{A>B} = (A_1 > B_1) + (A_1 = B_1)(A_0 > B_0) \quad F_{A=B} = (A_1 = B_1)(A_0 = B_0)$$

$$F_{A<B} = (A_1 < B_1) + (A_1 = B_1)(A_0 < B_0)$$

两位数值比较器逻辑图



3、4 位数值比较器：

4 位数值比较器功能表

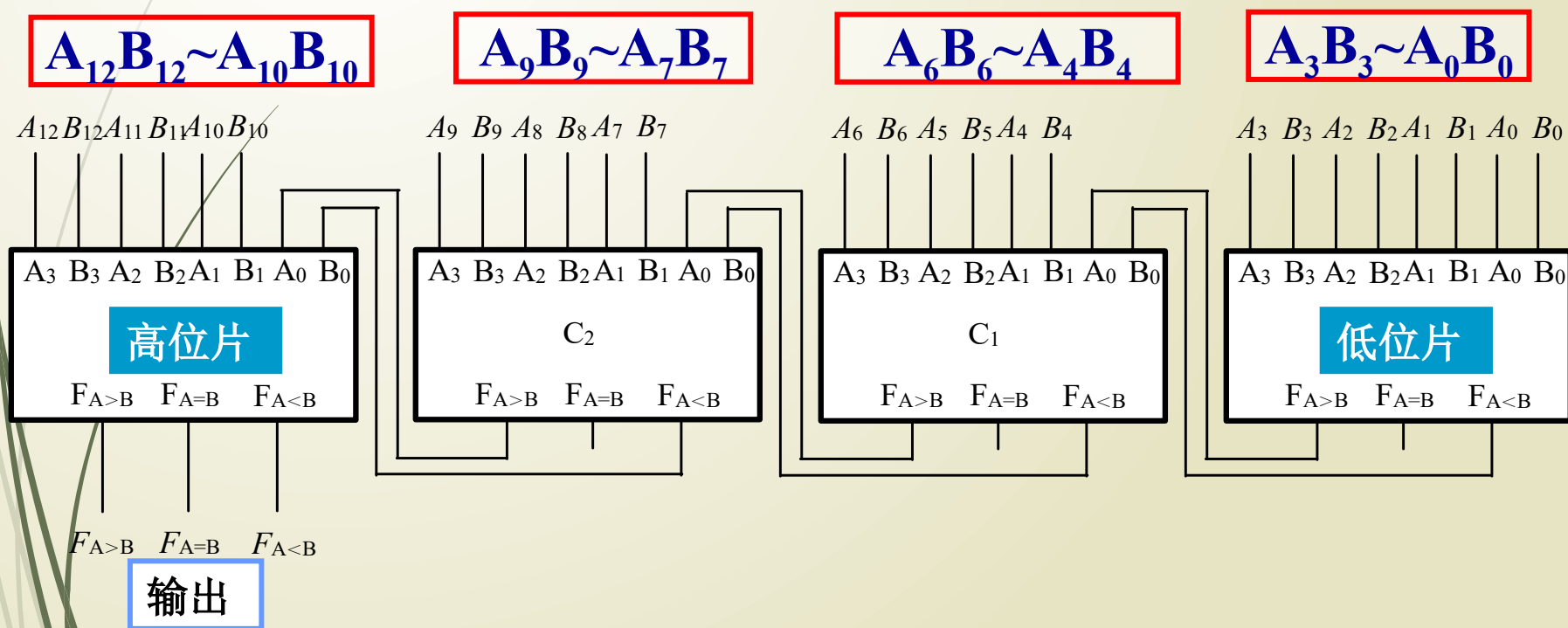
输 入				输 出		
$A_3 B_3$	$A_2 B_2$	$A_1 B_1$	$A_0 B_0$	$F_{A>B}$	$F_{A<B}$	$F_{A=B}$
$A_3 > B_3$	$\times$	$\times$	$\times$	1	0	0
$A_3 < B_3$	$\times$	$\times$	$\times$	0	1	0
$A_3 = B_3$	$A_2 > B_2$	$\times$	$\times$	1	0	0
$A_3 = B_3$	$A_2 < B_2$	$\times$	$\times$	0	1	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 > B_1$	$\times$	1	0	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 < B_1$	$\times$	0	1	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 = B_1$	$A_0 > B_0$	1	0	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 = B_1$	$A_0 < B_0$	0	1	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 = B_1$	$A_0 = B_0$	0	0	1

4、数值比较器的扩展

用四个 4 位数值比较器组成 13 位数值比较器 (**串联扩展方式**)

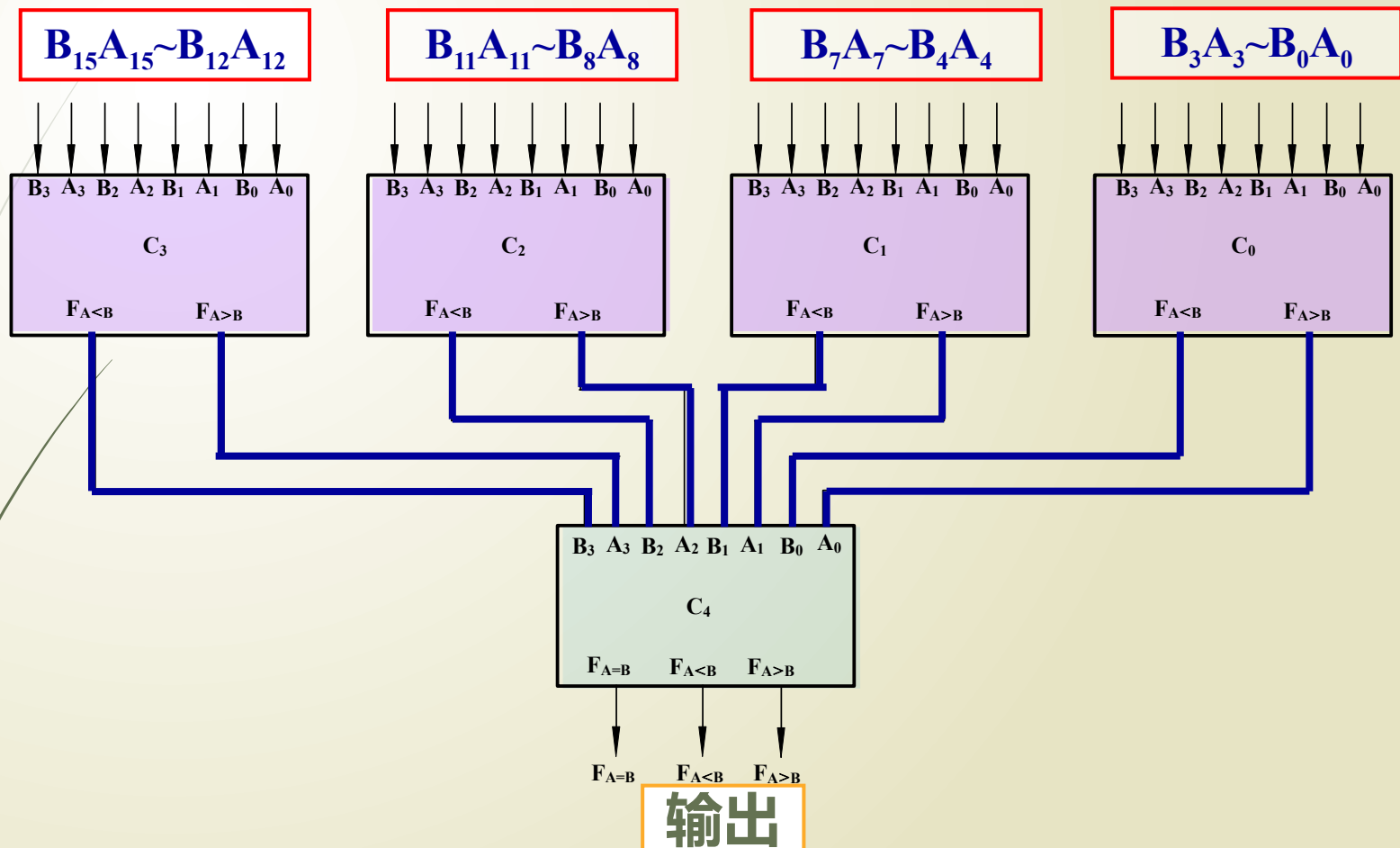
输入： $A=A_{12}A_{11}A_{10}\cdots A_3A_2A_1A_0$ $B=B_{12}B_{11}B_{10}\cdots B_3B_2B_1B_0$

输出： $F_{A>B}$ $F_{A<B}$ $F_{A=B}$



问题：如果每一片延迟时间为 10ns，四片串行比较器延迟时间？

用 4 位数值比较器组成 16 位数值比较器的**并联扩展**方式。



问题：如果每一片延迟时间为 10ns，16 位并行比较器延迟时间？

4.4.5 算术运算电路

1、半加器和全加器

两个 1 位二进制数相加时，不考虑低位来的进位的加法

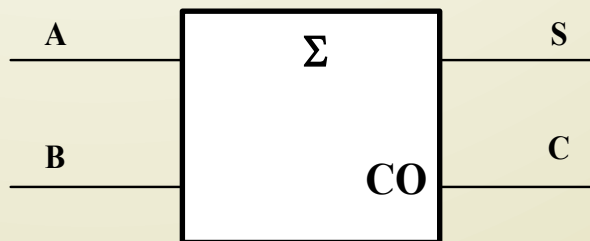
--- 半加

在两个 1 位二进制数相加时，考虑低位进位的加法

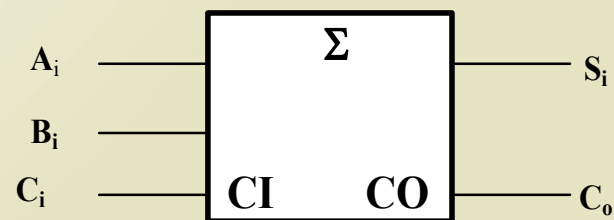
--- 全加

加法器分为半加器和全加器两种。

半加器



全加器



(1) 1 位半加器 (Half Adder)

不考虑低位进位，将两个 1 位二进制数 A、B 相加的器件。

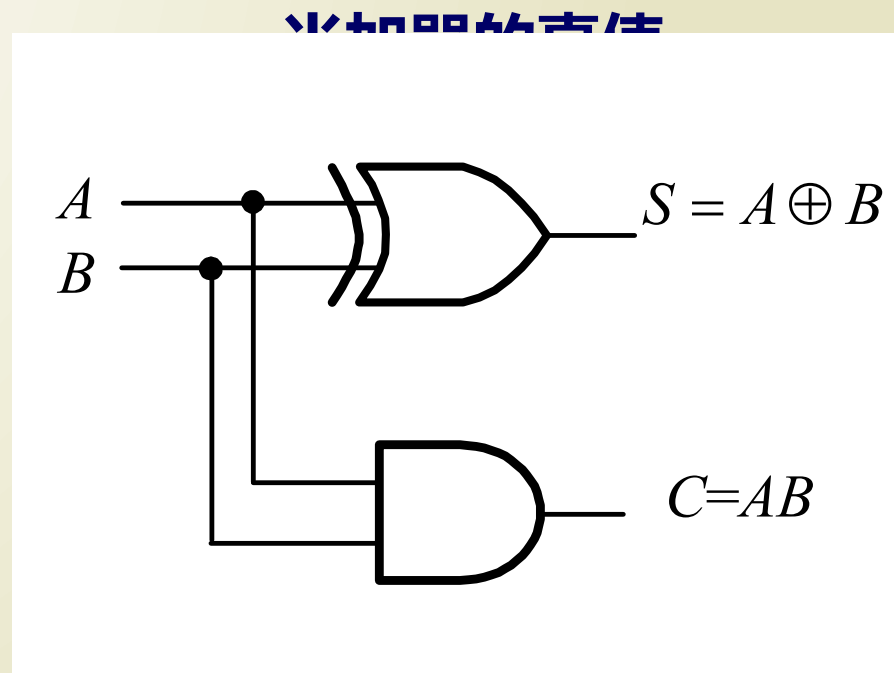
半加器的真值表

逻辑表达式

$$S = \overline{A}B + A\overline{B}$$

$$C = AB$$

如用与非门实现最少要几个门？



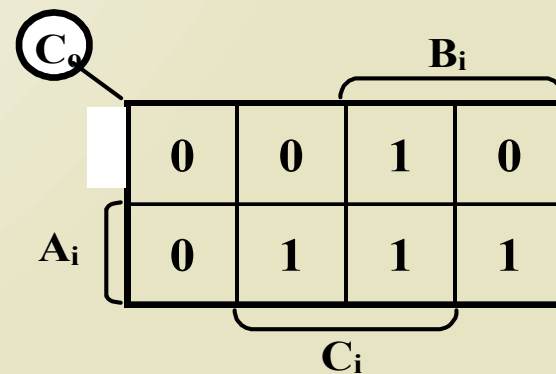
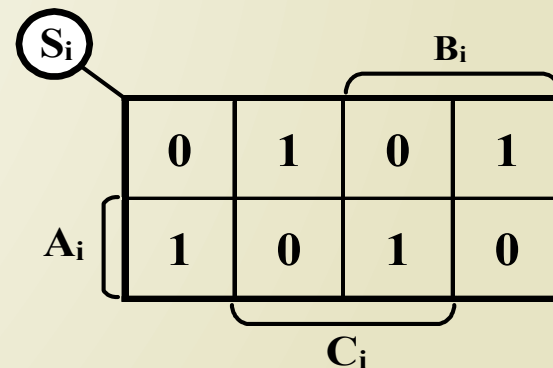
逻辑图

(2) 全加器 (Full Adder)

全加器能进行加数、被加数和低位来的进位信号相加，并根据求和结果给出该位的进位信号。

全加器真值表

A	B	C _i	S	C _o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



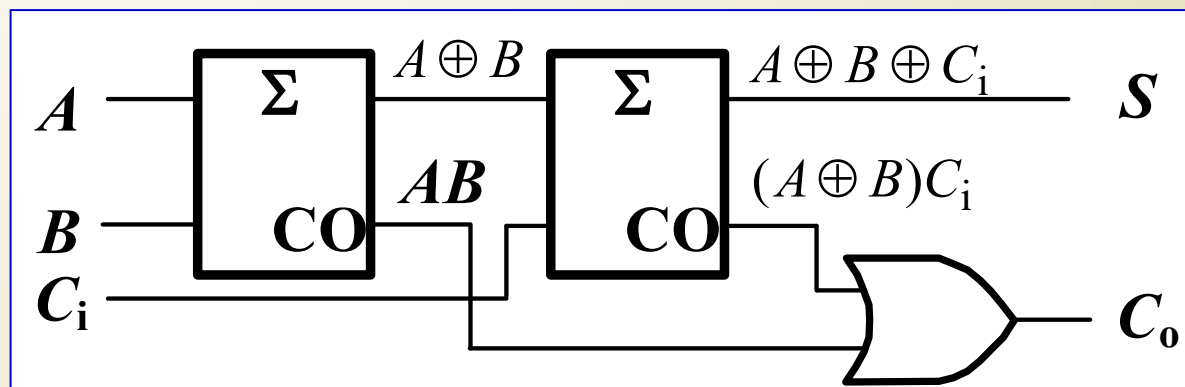
于是可得全加器的逻辑表达式为

$$S = \overline{A}\overline{B}C_i + \overline{A}B\overline{C}_i + A\overline{B}\overline{C}_i + ABC_i$$

$$= A \oplus B \oplus C_i$$

$$C_o = AB + \overline{A}\overline{B}C_i + \overline{A}BC_i$$

$$= AB + (A \oplus B)C_i$$



加法器的应用

全加器真值表

A	B	C_i	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

ABC_i 有奇数个 1 时 S 为 1 ;

ABC_i 有偶数个 1 和全为 0 时
 S 为 0 。

----- 用全加器组成三位二进制
代码奇偶校验器

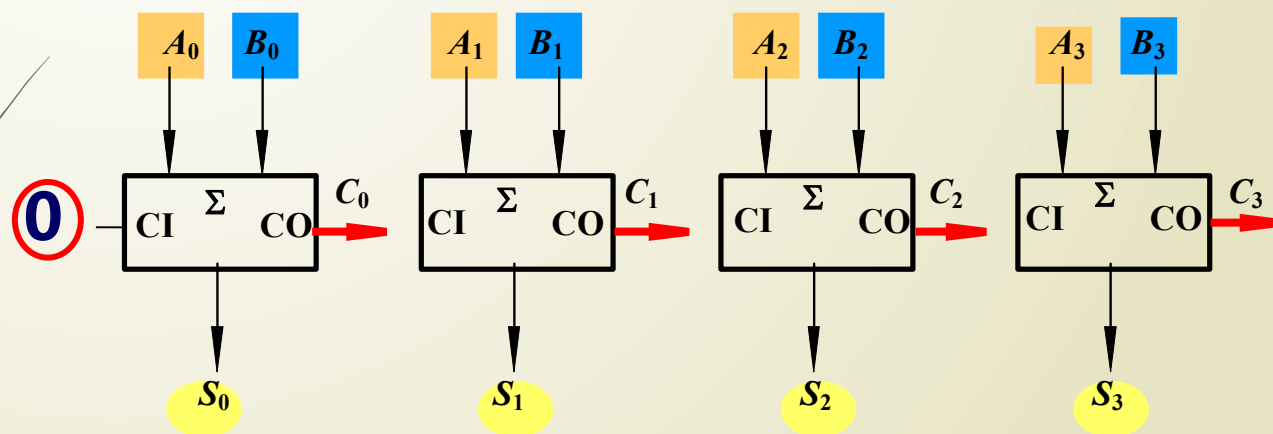
用全加器组成八位二进制代码
奇偶校验器，电路应如何连接？

2、多位数加法器

◆ 如何用 1 位全加器实现两个四位二进制数相加？

$$A_3 A_2 A_1 A_0 + B_3 B_2 B_1 B_0 = ?$$

(1) 串行进位加法器



◆ 低位的进位信号送给邻近高位作为输入信号，采用串行进位加法器运算速度不高。

(2) 超前进位加法器

提高运算速度的基本思想：设计进位信号产生电路，在输入每位的加数和被加数时，同时获得该位全加的进位信号，而无需等待最低位的进位信号。

定义第 i 位的进位信号 (C_i)：

$$C_i = A_i B_i + (A_i \oplus B_i) C_{i-1}$$

定义两个中间变量 G_i 和 P_i ：

$$G_i = A_i B_i$$

$$P_i = (A_i \oplus B_i)$$

$$C_i = G_i + P_i C_{i-1}$$

$$C_{i-1}$$

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

4 位全加器进位信号的产生:

由于 $C_i = G_i + P_i$

$$[G_i = A_i B_i \quad p_i = (A_i \oplus B_i)]$$

$$C_0 = G_0 + P_0 C_{-1}$$

$$C_1 = G_1 + P_1 C_0$$

$$C_1 = G_1 + P_1 G_0 + P_1 P_0 C_{-1}$$

$$C_2 = G_2 + P_2 C_1$$

$$C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{-1}$$

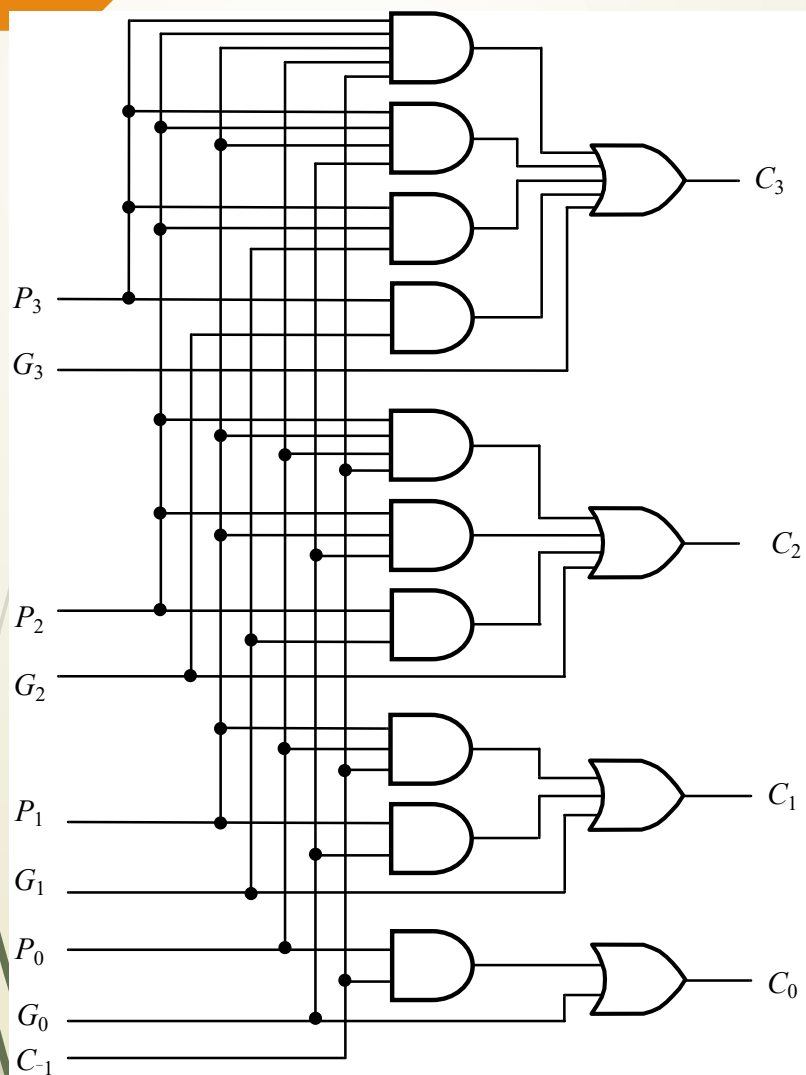
$$C_3 = G_3 + P_3 C_2 = G_3 + P_3 (G_2 + P_2 C_1) = G_3 + P_3 G_2 + P_3 P_2 C_1$$

$$= G_3 + P_3 G_2 + P_3 P_2 (G_1 + P_1 C_0)$$

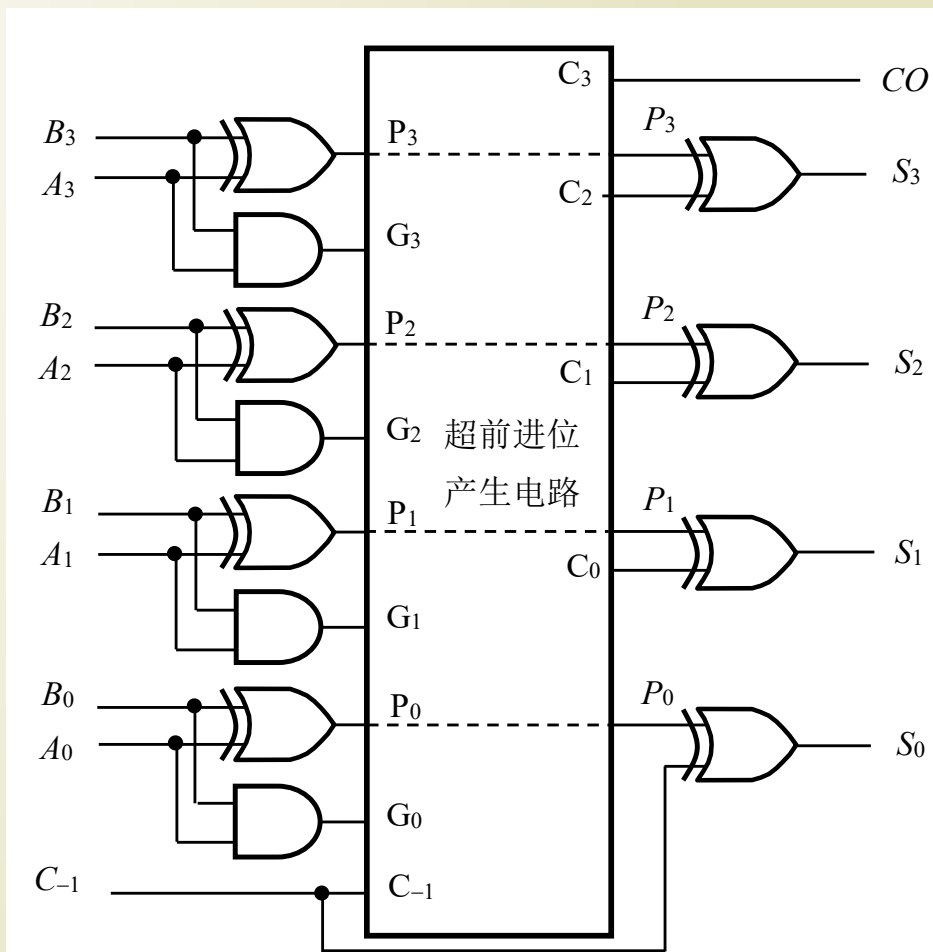
$$C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 (G_0 + P_0 C_{-1})$$

进位信号只由被加数、加数和 C_{-1} 决定，而与其它低位的进位无关。提高了速度，但位数增加时，进位电路复杂度增加。

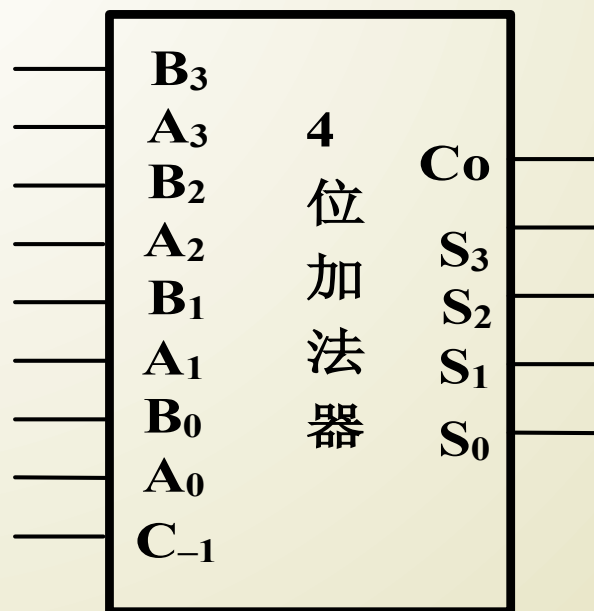
超前进位产生电路



4 位超前进位加法器

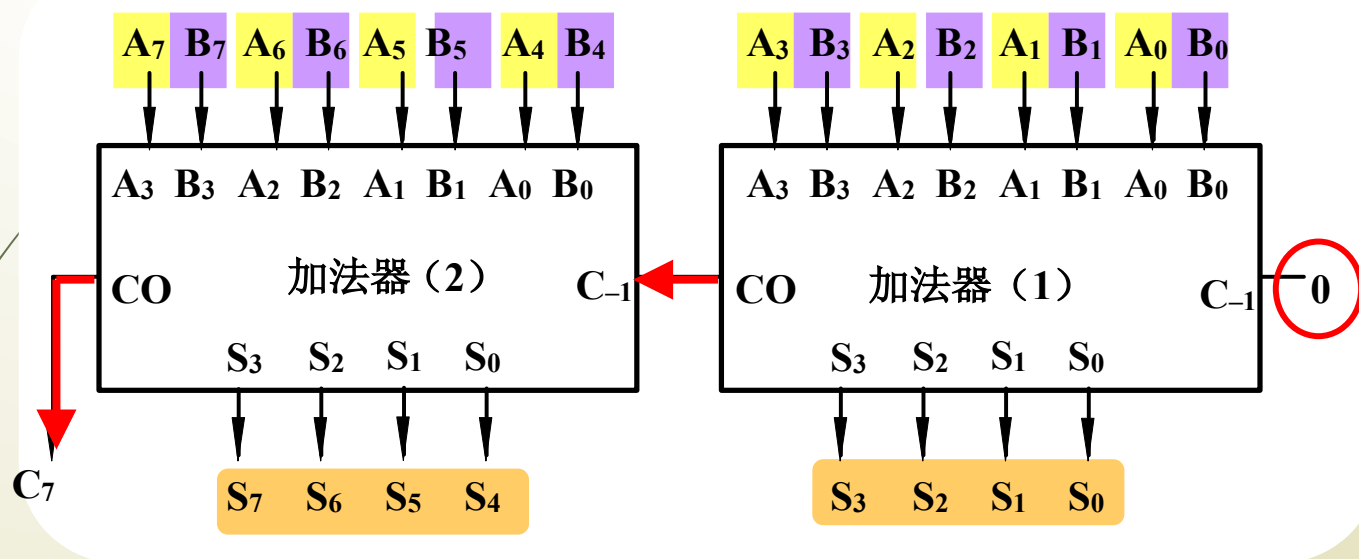


4 位加法器框图



(3) 4 位超前进位加法器的应用

例 1. 用两片 4 位加法器构成一个 8 位二进制数加法器。

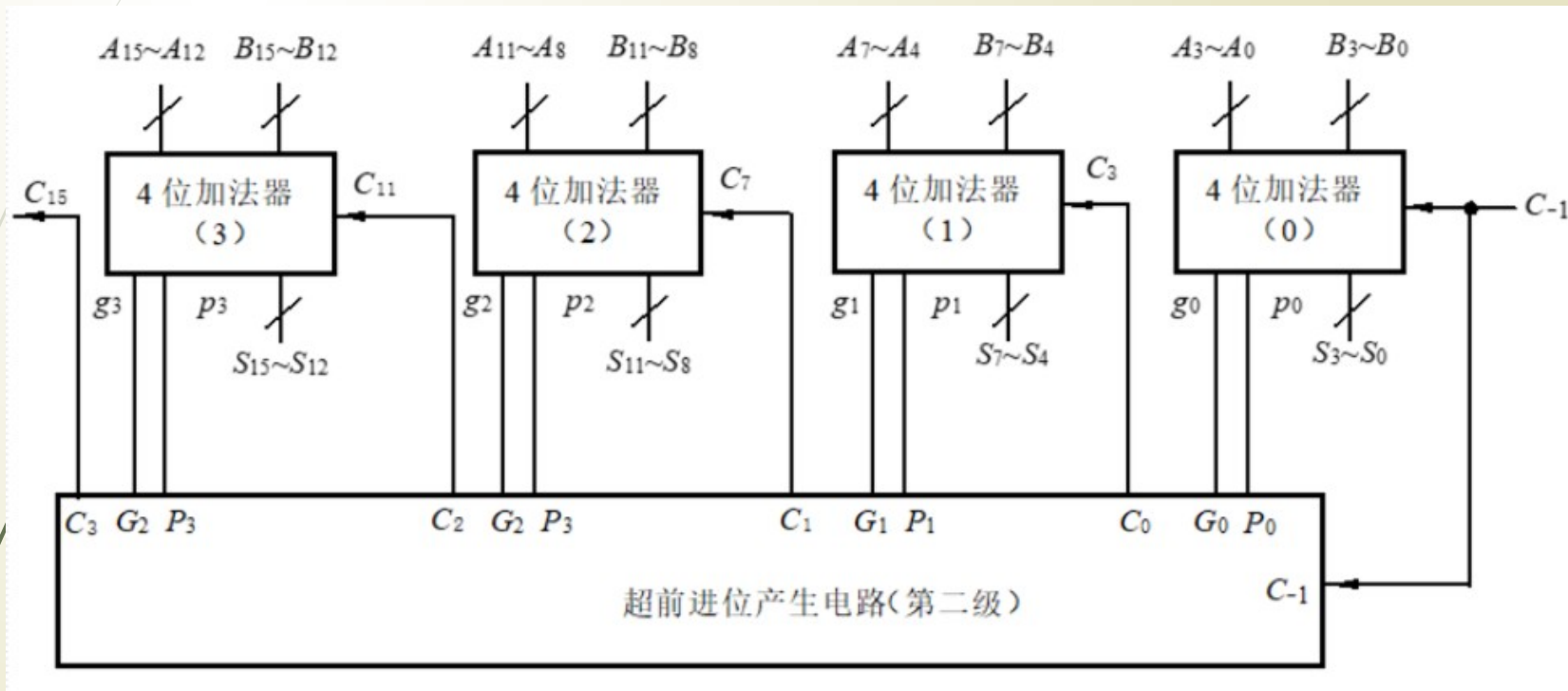


在片内是超前进位，而片与片之间是串行进位。

问题：如果每一片延迟时间为 $4t_{pd}$ ，4 片串行进位延迟时间？

(3) 4位超前进位加法器的应用

例 2 用四片 4 位加法器构成一个 16 位二进制数加法器。

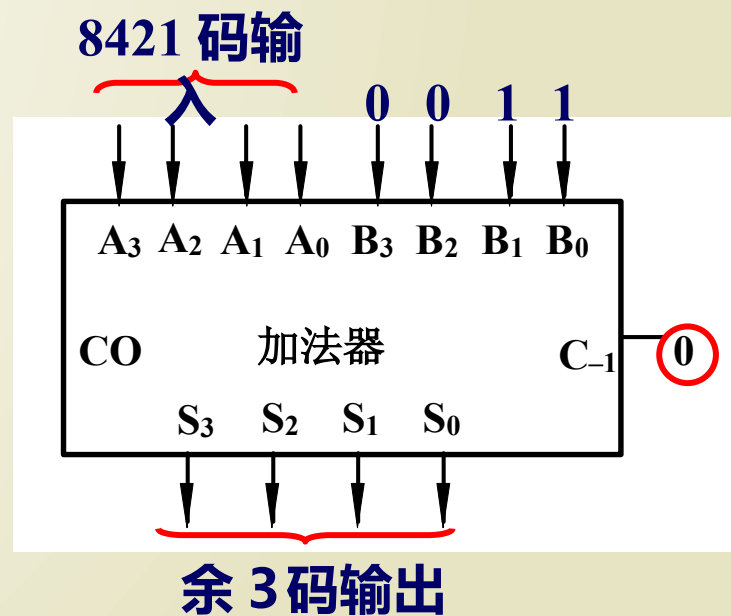


在片内是超前进位，而片与片之间是超前进位。

延迟问题：共 8 级门延迟 (书 P193) $8t_{pd}$ 。当级数增加，延迟不增加。

例．用 4 位加法器构成将 8421BCD 码转换为余 3 码的码制转换电路。

8421 码		余 3 码
0000	+0011	0011
0001	+0011	0100
0010	+0011	0101



3、减法运算

若 n 位二进制的原码为 $N_{\text{原}}$ ，则与它相对应的 2 的补码为

$$N_{\text{补}} = 2^N - N_{\text{原}}$$

补码与反码的关系式

$$N_{\text{补}} = N_{\text{反}} + 1$$

设两个数 A 、 B 相减，利用以上两式

可得

$$A - B = A + B_{\text{补}} \square 2^n = A + B_{\text{反}} + 1 - 2^n$$

在实际应用中，通常是将减法运算变为加法运算来处理，即采用加补码的方法完成减法运算。

1) $A-B \geq 0$ 的情况。

2) $A-B < 0$ 的情况。

$A=0101$,
 $B=0010$

$$\begin{array}{rcccccl} & \boxed{0} & 1 & 0 & 1 & A \\ & \boxed{1} & 1 & 0 & 1 & B_{\text{反}} \\ + & & & & & 1 \\ \hline 1 & \boxed{0} & 0 & 1 & 1 & \end{array}$$

舍弃

当 $A-B \geq 0$ 时，舍弃的进位为 1，所得结果就是差的原码，不需再求反补。

$A=0010$, $B=0101$

$$\begin{array}{rcccccl} & \boxed{0} & 0 & 1 & 0 & A \\ & \boxed{1} & 0 & 1 & 0 & B_{\text{反}} \\ + & & & & & 1 \\ \hline 0 & \boxed{1} & 1 & 0 & 1 & \end{array}$$

舍弃

当 $A-B < 0$ 时，舍弃的进位为 0，所得结果是补码，要得到原码需再求补。

输出为原码的 4 位减法运算逻辑

